LEARN · VERIFY — OFFLINE STUDY BOOKLET

Large-Scale Change & Deprecation

A decision is only permanent if you cannot change every use of it. Here is why the one big commit stops being possible, who has to pay for the alternative, the machine that does the work — and how to decide what to remove and make the removal actually finish.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

1. Read the plain-version box first. If it doesn't land, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

> **THE RULE THAT DOES MOST OF THE WORK**
>
> If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

> **WHAT THIS SUBJECT ASKS OF YOU THAT OTHERS DON'T**
>
> Most engineering subjects are about making something work. This one is about **the cost of having already made something work** — and it keeps handing you conclusions that feel backwards. The bigger your codebase, the *smaller* the change you can make in one go. The team that should do a migration is the one that did not cause the problem. The right response to code you are proud of may be to delete it. Almost none of this is a technical difficulty; it is a question of who bears a cost that is spread thinly across everybody. So as you read, keep asking the question this subject is really made of: *whose budget does this land on, and what happens if the answer is “nobody's”?*

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

What's in here

SECTION 1

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Faced with a change that touches more code than one commit can hold, or an obsolete system nobody has removed, you can:

1. **Say why the atomic version is not available**, naming which of the five barriers applies to your situation rather than asserting that it is “too big”.
2. **Put the cost where the expertise is**, and explain why leaving the migration to each consuming team is both slower and less likely to finish.
3. **Describe the four phases of the process** and the two things that make it scale — change generation whose human effort does not grow with the codebase, and review that does not grow with the number of shards.
4. **Choose between advisory and compulsory deprecation**, and say what each one requires from you before it is honest to start.

The core model in one paragraph

Everything here follows from one observation that runs the opposite way to intuition: **as a codebase and the number of engineers in it grow, the largest atomic change you can make gets smaller**. Running every affected check becomes difficult; merge conflicts become certain; some corners of the code are too frightening to touch at all. So the sweeping change has to be broken into many small independent ones — which is a harder thing to execute, and only worth doing if a machine does the work. Building that machine turns out to be mostly a social problem rather than a technical one. Someone has to fund it, because the benefits of a migration are spread thinly across everybody and the costs are not; the team that should do it is the one holding the expertise, which is the team that built the thing being migrated *away from*; and the reviewers have to accept that a change nobody on their team wrote is going to land in their code. Once that machine exists, something unexpected becomes true: **technical decisions stop being permanent**. A widely used name, the location of a class, the choice of an idiom — these stop being one-way doors. And that changes the last question, which is not *how* to remove an obsolete system but *whether* to: because code is a liability rather than an asset, and the honest comparison is between the cost of keeping it and the cost of removing it.

TENTH-GRADER VERSION — THE WHOLE THING AT ONCE

- Some changes touch so much code that you cannot put them in one commit. Not “shouldn't” — *cannot*.
- Strangely, the bigger the project gets, the smaller that one-commit limit becomes.
- So instead you make thousands of tiny changes. That only works if a program writes them, not a person.
- The hard part isn't the program. It's that nobody wants to pay for a job that helps everyone a little and nobody a lot.
- The team that should do it is the team that owns the old thing — because they know it, and because they're the ones who want it gone.
- Once you can change every use of something, decisions you thought were permanent turn out not to be.
- And then the real question is what to delete. Code isn't treasure; it's a bill that arrives every month.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 Why one big commit stops working	What a large-scale change actually is; the counterintuitive shrinking of the atomic limit; and the five barriers — version-control limits, merge conflicts, code nobody dares touch, an inconsistent environment, and testing that gets harder as the change gets bigger.	The whole discipline exists because of a constraint. Naming which barrier you are actually hitting is what tells you whether to shard, to test, or to simplify your environment first.
2 Who pays for the migration	Why the infrastructure team does the work rather than each consuming team; unfunded mandates; why organic migrations do not finish; the small fixes that only become possible once the machinery exists; and treating individual changes as replaceable rather than precious.	The barriers in Part 1 are technical, but the reason migrations stall is economic. This part is the one that decides whether anything actually happens.
3 The machine that does it	The four phases — authorisation, change creation, shard management, cleanup; the oversight committee and what it is for; change generation whose effort scales sublinearly; sharding and the independent test-mail-submit pipeline; batching tests on a train; and review by a global approver.	Now that you know the constraint and who is paying, this is what the money buys. Each piece exists to answer one barrier from Part 1 or one incentive problem from Part 2.
4 Deciding what to remove	Code as a liability; why removal is the hardest change of all; designing systems so they can be decommissioned; advisory versus compulsory deprecation; warnings that are actionable and relevant; and why a deprecation without an owner never finishes.	Last, because it is the part the machine serves. A migration is a means; deciding what deserves removing — and committing to finish it — is the judgement the machine cannot make for you.

> *“An LSC process makes it possible to rethink the immutability of certain technical decisions.”*

1

PART 1 OF 4

Why one big commit stops working

What a large-scale change is, and the five things that make the atomic version impossible.

TENTH-GRADER VERSION

- Ask yourself: how many files could you change in one commit right now? Could you do it in an emergency? Could you undo it?
- The answer gets *worse* as the project grows, which is not what anyone expects.
- Five things get in the way. The version control system runs out of room. Somebody else edits a file while you're working. Some code is so scary nobody will touch it. Every project does things slightly differently, so a robot can't handle them all. And the bigger the change, the harder it is to work out which bit broke the test.
- Finding the one broken file out of 25 is easy. Out of 10,000 it is not.
- The cure for the scary code is boring: tests. Test it well enough and its age stops mattering.
- Splitting the change into small pieces gets round all of this — and makes the job more complicated to run.

THE EVERYDAY VERSION

A city wants to renumber every house on every street, because the old scheme has become a mess. On a village lane you do it in an afternoon: knock on eight doors, agree it, done. In a city of two million you cannot do it at all in one go — and notice that the reason is not that the job is harder per house. It is that while you are working, people keep moving in and out, so any list you compile is out of date before you finish it; a few buildings are listed and legally frozen and nobody will authorise touching them; the boroughs each keep their records in a different format, so one clerk cannot process them all; and if something goes wrong afterwards, you have no way of telling which of two million changes caused it. So you do it street by street, over a year. That is slower and far more complicated to administer than the afternoon in the village — and it is the only version that exists.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Large-scale change (LSC)	Any set of changes that are **logically related but cannot practically be submitted as a single atomic unit**.	It may touch so many files that the tooling cannot commit them all at once, or be so large that it would always have merge conflicts. In some cases it is dictated by repository topology: across a collection of distributed or federated repositories, an atomic change may not even be technically possible.	vs a big change. The defining property is not size but **the impossibility of atomicity** — which is why the same process scales down to a team making only a few dozen related changes. These changes are almost always generated by automated tooling.

The counterintuitive shrink	As a codebase and the number of engineers working in it grow, **the largest atomic change possible counterintuitively decreases**.	Running all affected presubmit checks and tests becomes difficult — to say nothing of ensuring that every file in the change is still up to date at the moment of submission.	vs the expectation that a bigger organisation can do bigger things. Everything else here is a response to this one fact, which is why the idea of making sweeping changes as large atomic commits was abandoned rather than optimised.
What they are for	Four basic categories: **cleaning up common antipatterns** using codebase-wide analysis tooling; **replacing uses of a deprecated library feature**; **enabling low-level infrastructure improvements** such as compiler upgrades; and **moving users from an old system to a newer one**.	Broader causes sit behind these: a new language standard introducing a more efficient idiom, an internal interface changing, or a new compiler release requiring fixes for things it will now flag as errors.	The **majority have near-zero functional impact** — widespread textual updates for clarity, optimisation or future compatibility. But they are **not theoretically limited** to this behaviour-preserving, refactoring class of change.
Barrier 1 — technical limits	Most version control systems have **operations that scale linearly with the size of a change**.	A system may handle commits on the order of tens of files perfectly well and lack the memory or processing power to atomically commit thousands. In centralised systems, commits can block other writers — and in older systems, readers — while they are processed, so large commits stall everyone else.	vs a matter of judgement. It might not be merely “difficult” or “unwise” to make a large change atomically: **it might simply be impossible** with a given set of infrastructure. Splitting it up gets around the limit at the cost of a more complex execution.
Barrier 2 — merge conflicts	Every version control system requires updating and merging, potentially with manual resolution, if a newer version of a file exists centrally. As the number of files in a change increases, so does the probability of a conflict — **compounded by the number of engineers working in the repository**.	A small company might sneak in a change touching every file over a weekend when nobody is developing, or pass a virtual — or even physical — token around the team to hold a global lock.	Neither trick survives scale: at a large, global company **somebody is always making changes to the repository**. The general property that follows holds for the other barriers too: with few files in a change, the probability of conflict shrinks, so it is more likely to be committed without problems.
Barrier 3 — haunted graveyards	A system so **ancient, obtuse, or complex that no one dares enter it**. The phrase comes from a production-engineering mantra: *no haunted graveyards*.	They are often business-critical systems frozen in time because any attempt to change them could make them fail in incomprehensible ways, costing real money. They pose a real existential risk and can consume an inordinate amount of resources.	They exist in codebases too, not only in production: old, unmaintained software written by someone long off the team, on the critical path of important revenue-generating functionality, with layers of bureaucracy built up to prevent destabilising changes. **The counter is good, old-fashioned testing:** thoroughly tested software can be changed arbitrarily and you will know whether the change breaks it, **no matter the age or complexity of the system**.

Barrier 4 — heterogeneity	Large-scale changes **work only when the bulk of the effort can be done by computers, not humans** — and computers rely on consistent environments to apply the right transformation in the right place.	Many different version control systems, continuous-integration systems, project-specific tooling or formatting guidelines all make sweeping changes difficult. Presubmit checks can be very complex, from checking new dependencies against a list, to running tests, to requiring an associated bug.	As good as humans are with ambiguity, machines are not — so **simplifying the environment helps the humans moving around in it as well as the robots**. The response is two-sided: embrace some of the complexity by making it standard everywhere, and ask teams to omit their special checks where a large-scale change touches their code.
Barrier 5 — testing	Every change should be tested, but **the larger the change, the more difficult it is to test appropriately**.	A continuous-integration system that runs not only the tests immediately affected but every test that **transitively depends** on the changed files gives broad coverage — and the farther away in the dependency graph a test is, the more unlikely a failure is to have been caused by the change itself.	The decisive comparison: finding the root cause of a test failure in a change of **25 files** is straightforward; finding one in a **10,000-file** change is the proverbial needle in a haystack. The trade-off accepted in exchange is that smaller changes rerun the same tests repeatedly — deliberately, because engineer time spent tracking down failures is **much more expensive than the compute time**. That balance may not hold for every organisation.

CONCEPT BOUNDARY — “TOO BIG TO COMMIT” IS A PROPERTY OF YOUR INFRASTRUCTURE, NOT OF YOUR CHANGE

What it sounds like: a judgement about the change — it is sprawling, it is risky, it should have been broken up by a more disciplined author.
What it actually is: a statement about the environment the change has to pass through. The same edit is atomic in one repository and impossible in another, and the thing that moved was never the edit.
Why that matters practically: it tells you where to spend. If the barrier is the version control system, sharding is the answer. If it is merge conflicts, smaller shards are the answer. If it is code nobody dares touch, **the answer is tests**, and no amount of tooling substitutes. If it is heterogeneity, the answer is consistency, and it pays back for humans as well as machines.
The trap: treating “we can't do that here” as permanent. Every one of the five barriers is something an organisation chose, inherited, or failed to invest in — and each of them can be moved.

Anchor sketch — the shrinking commit, and what stands in the way

Legend: bold = a named barrier or term **reversed = the fact everything else answers** **Q = cover the page and answer this**

```
AN LSC = changes that are LOGICALLY RELATED but cannot
practically be submitted as a SINGLE ATOMIC UNIT.
     not "big". "not atomic". the process scales all the
     way down to a few dozen related changes.
                       |
AS CODEBASE AND HEADCOUNT GROW, THE ATOMIC LIMIT SHRINKS
     running every affected check gets hard, and every
     file must still be up to date AT submission.
                       |
FOUR THINGS THEY ARE FOR
     clean up antipatterns . replace a deprecated library
     feature . enable low-level infra work (e.g. compiler
     upgrades) . move users off an old system
     +--> MOST have near-zero functional impact. but they
          are not limited to refactoring.
                       |
FIVE BARRIERS TO DOING IT ATOMICALLY
-----------------------------------------------------
TECHNICAL LIMITS . VCS operations scale LINEARLY with
     change size. tens of files fine; thousands may be
     impossible. commits can block other writers.
     -> not "unwise". IMPOSSIBLE. shard it.
MERGE CONFLICTS . probability grows with files AND
     with engineers. the weekend trick and the passed
     token do not survive scale: somebody is ALWAYS
     committing. -> fewer files per change.
HAUNTED GRAVEYARDS . a system so ancient, obtuse or
     complex that NO ONE DARES ENTER IT. often business-
     critical, frozen, wrapped in bureaucracy.
     -> the counter is TESTING. test it well and its age
        and complexity stop mattering.
HETEROGENEITY . robots need consistency. many VCSs,
     CI systems, per-project tooling and formats make
     sweeping change impossible.
     -> standardise what you can; ask teams to omit
        special checks for LSC shards.
TESTING . CI runs everything that TRANSITIVELY depends
     on the change. the farther out in the graph, the less
     likely a failure is really yours.
     -> root-causing 1 bad file in 25 is easy.
        in 10,000 it is a needle in a haystack.
     -> price accepted: small changes rerun the same
        tests. engineer time >> compute time.
```

Q Define a large-scale change without using the word “big”.

Q Name the five barriers, and the specific remedy each one has.

Q Which barrier does testing *solve*, and which one does testing *make worse*?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN

Splitting a large change into small independent chunks	It gets around the version-control limits, and shrinks the probability of merge conflicts.	The execution of the change becomes more complex.	Whenever the atomic version is impossible — which, past a certain size, is always.
Testing everything that transitively depends on a change	Broad coverage: nothing downstream goes unchecked.	High load on the CI system, and failures far out in the graph that are unlikely to be yours.	When you would rather over-test than ship a silent break — and can afford the compute.
Many small changes rather than one large one	Failures are easy to diagnose, and each change affects a smaller set of tests.	The same tests get run repeatedly, especially ones depending on large parts of the codebase.	When engineer time costs more than compute time. That is a conscious decision, and it may not hold for you.
Writing tests for the code nobody dares touch	The only thing that dissolves a haunted graveyard — arbitrary changes become safe regardless of the system's age.	A lot of effort, on software nobody wants to work on.	Whenever a frozen system is blocking migrations, decommissioning, or a compiler or library upgrade.
Standardising checks and formats across projects	Robots can operate everywhere, and humans can move between projects more easily.	Teams lose local rules they chose deliberately.	When heterogeneity is what is stopping automation — and note the human benefit is the same benefit.
Investing in change tooling early	It pays off for the many teams doing infrastructure work — and the same tools scale down to tens of files.	Engineering time spent on tooling rather than on the changes themselves.	Early and often. The tools that make thousands of files efficient work reasonably well on tens.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“That change is too big.”	“It isn't atomically submittable here. Which barrier are we hitting — the VCS, conflicts, or the test graph?”
“We'll be able to do bigger changes once we're a bigger team.”	“The opposite. The atomic limit shrinks as the codebase and the headcount grow.”
“We'll grab the repo on a quiet weekend.”	“That works while somebody isn't always committing. Plan for the version where they are.”
“Nobody touches that service, it's too risky.”	“That's a haunted graveyard, and it will block every migration that goes near it. The way out is tests, not caution.”
“Each team has its own presubmit rules.”	“Then automated change is off the table. Consistency buys the robots and the humans the same thing.”
“A test failed somewhere downstream.”	“How far out in the dependency graph? The farther it is, the less likely it's caused by this change.”
“Running these tests over and over is wasteful.”	“It's a deliberate trade: compute is cheaper than an engineer hunting a failure through 10,000 files.”

PART 1 · QUESTION 1

A platform team needs to rename a configuration key used in roughly 40,000 files across one repository. Their first attempt — a single commit — was rejected by the version control server after twenty minutes. Their second attempt succeeded locally but failed to land because eleven files had changed underneath them. A colleague suggests scheduling the third attempt for 3 a.m. on a public holiday. Separately, about 300 of the affected files belong to a billing service written six years ago whose last maintainer left the company; the team's own instinct is to skip those files and file a ticket.

(a) Define a large-scale change, and say whether this qualifies and on what grounds. **(b)** Name the barrier behind each of the two failed attempts, and give the stated remedy for each. **(c)** Assess the 3 a.m. proposal: say what it is a version of, and the precise condition under which it stops working. **(d)** Name what the billing service is, say why skipping it is worse than it looks, and give the one thing that dissolves it.

PART 1 · QUESTION 2

Two migrations are proposed in the same quarter. The first replaces a logging call across 9,000 files; the team plans one change and reports that a test three layers away in the dependency graph failed, which they have spent two days investigating. The second is the same edit across 60 files in a subsystem where every project uses a different formatter, a different CI configuration and two different version control systems. A director asks why the second one — a fiftieth of the size — is being quoted as the harder job, and proposes that both be split into 200-file commits to “standardise the approach”.

(a) Explain the first team's two days using what is said about the dependency graph, and say what change to their approach would have prevented it. **(b)** Answer the director's question: name the barrier the second migration is hitting and why size does not govern it. **(c)** Evaluate the 200-file proposal for each migration separately. **(d)** Splitting has a stated cost and produces a stated inefficiency. Name both, and give the reasoning by which the inefficiency is accepted.

2×2 — how safe is it to change, and can a machine do it

COLUMNS: IS THE ENVIRONMENT CONSISTENT ENOUGH TO AUTOMATE? →

	Yes — one set of tools, checks and formats	**No — every project does it differently**
The code is THOROUGHLY TESTED	**Malleable** Arbitrary changes can be made and you will know whether they broke anything, whatever the age or complexity of the system — and a machine can apply them everywhere. *Move:* this is the cell the whole subject is trying to reach, and it is worth naming what it buys: technical decisions stop being permanent. Protect it by keeping the environment standard and by treating flaky tests as a cost borne by whoever writes them.	**Safe to change, expensive to change** You could alter anything without fear, and the alteration has to be done by hand in each project because no single tool understands them all. *Move:* spend on consistency, not on courage. Standardise what you can and ask teams to omit their special checks for these shards — and note the humans moving between projects get the same benefit the robots do.
NOT really tested	**The robots can drive, and nobody dares let them** The tooling would work. What is missing is any way to know whether a generated change broke something, so every change is a gamble. *Move:* write the tests. This is the unglamorous answer and it is the only one; without them, automation multiplies risk rather than reducing it.	**The haunted graveyard** Ancient, obtuse or complex enough that no one dares enter, often business-critical and frozen, with bureaucracy layered on to prevent change. *Move:* understand that this cell blocks other people's work — migrations cannot complete, dependent systems cannot be decommissioned, compilers and libraries cannot be upgraded. It is not a local decision to stay here.

Read it as: the row decides whether a change is *safe* and the column decides whether it is *affordable*, and they fail in different ways — an untested consistent codebase is a fast way to break things, and a tested inconsistent one is a slow way to fix them. Only the top-left keeps a codebase responsive to changes in the infrastructure underneath it.

CARRY-OUT RULE FROM THIS PART

When someone says a change is too big, ask which barrier they mean. There are five, they have different remedies, and the word “big” hides all of them. Version-control limits and merge conflicts are answered by sharding; frightening code is answered by tests; an inconsistent environment is answered by standardisation; and hard-to-diagnose failures are answered by making each change smaller, not by testing less.

2

PART 2 OF 4

Who pays for the migration

Why the work lands centrally, and what happens to it when it does not.

TENTH-GRADER VERSION

- The obvious plan is: build the new thing, tell everyone to move. It doesn't work.
- The people who know how to do the migration are the people who built the thing being migrated away from. Everyone else would have to learn it — separately, hundreds of times over.
- If the benefit is spread thinly across everybody, no single team will ever make it their priority. That is called an unfunded mandate, and nobody likes one.
- People write new code by copying old code, so an old system keeps producing new uses of itself.
- So the team that wants the old thing gone should be the team that removes it. They want it finished, and they already know how.
- Once you have the machinery, you can fix tiny irritations that were never worth fixing before.
- And you stop being precious about individual commits: a machine made them, a machine can make more.

THE EVERYDAY VERSION

A city changes its recycling rules, and someone has to relabel every bin. Option one: send every household a letter saying the rules have changed and please relabel your own bin. Each household then has to work out what the new rules mean, find the right labels, and do a small job badly and differently from their neighbours; most will not bother, because a slightly better recycling rate next year is worth almost nothing to any one of them. Option two: the department that changed the rules sends a crew round with the correct labels and a van. The crew does the first bin slowly, the next hundred quickly, and by the end they have seen every awkward case twice and know exactly what to do about it. Option two costs the department real money that option one appeared to cost nobody — and that appearance is the entire problem, because the cost in option one was real, it was just spread across two million people who each decided it wasn't their job.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Why the infrastructure team owns it	The teams that build and manage the underlying systems are **also the ones with the domain knowledge required to fix the hundreds of thousands of references to them**.	Teams that consume the infrastructure are unlikely to have the context for handling many of these migrations, and it is **globally inefficient** to expect them each to relearn expertise the infrastructure team already has.	vs the intuitive plan — introduce the new thing and dictate that everyone using the old one moves. That seems easier and **turns out not to scale**. Centralisation also allows faster recovery from errors, because errors generally fall into a small set of categories and the team running the migration can keep a playbook — formal or informal — for handling them.

The repeated first change	The cost of doing the first of a series of semi-mechanical changes you do not understand.	You spend time reading about the motivation and nature of the change, find an easy example, try to follow the provided suggestions, and then try to apply that to your local code.	**Repeating this for every team in an organisation greatly increases the overall cost of execution.** Making a few centralised teams responsible both internalises those costs and drives them down, because the change can then happen more efficiently.
Unfunded mandate	“Additional requirements imposed by an external entity without balancing compensation.” And **nobody likes them**.	Even where a new system is categorically better than the one it replaces, those benefits are **often diffused across an organisation** and thus unlikely to matter enough for individual teams to want to update on their own initiative.	vs the costs disappearing. **If the new system is important enough to migrate to, the costs of migration will be borne somewhere in the organisation.** Centralising the migration and accounting for its costs is almost always faster and cheaper than depending on individual teams to migrate organically.
Why organic migrations stall	They are **unlikely to fully succeed**, in part because **engineers tend to use existing code as examples when writing new code**.	So the old system keeps generating new uses of itself for as long as it exists, regardless of what the documentation says.	The structural answer is incentive alignment: having a team with **a vested interest in removing the old system** responsible for the migration effort helps ensure it actually gets done. Funding and staffing such a team looks like an additional cost, and is **actually just internalising the externalities that an unfunded mandate creates**, with the added benefit of economies of scale.
Filling potholes	The small fixes that become possible once the machinery exists, and **simply wouldn't have been possible without it**.	Two header files in the same in-house library were named with **different word separators** — one with an underscore, one with a hyphen. Beyond irritating the consistency purists, engineers had to remember which file used which, and only discovered they had got it wrong **after a potentially lengthy compile cycle**.	Fixing a single-character difference across a codebase of that size sounds pointless. Mature tooling made it **a couple of weeks' worth of background-task effort**; library authors could apply it en masse **without having to bother end users of those files**; and it **quantitatively reduced the number of build failures** caused by that specific issue. The resulting productivity — and happiness — more than paid for the time.
Decisions stop being immutable	As the ability to change the entire codebase improves, **the diversity of changes expands**, and engineering decisions can be made knowing they aren't fixed forever.	Technical decisions that used to be final — the name of a widely used symbol, or the location of a popular class within a codebase — **no longer need to be final**.	This is the payoff the rest of the machinery buys, and it works backwards in time: it changes what you are willing to decide **at design time**, not just what you can fix later. Sometimes it is worth the effort to fill a few potholes.
Cattle, not pets	The stance to take toward the individual changes a large migration produces.	A typical change is handcrafted: an engineer may spend days or weeks on the creation, testing and review of a single one, comes to know it intimately, and is proud when it is committed. That is **owning and raising a favourite pet**.	Effective large-scale change requires a high degree of automation and produces an enormous number of individual changes. Here it helps to treat them as **cattle: nameless and faceless commits that might be rolled back or otherwise rejected at any time with little cost, unless the entire herd is affected.** Rejection of a pet is hard not to take personally; with automation, tooling can be updated and new changes generated at very low cost, so **losing a few cattle now and then isn't a problem**.

The social contract	Code owners need a sense of responsibility for their software *and* an understanding that these migrations are part of how the organisation scales its engineering.	Product teams are most familiar with their own software; library infrastructure teams know the nuances of the infrastructure. **Getting product teams to trust that domain expertise is an important step toward social acceptance.**	The line that is easy to get wrong: owners **should understand the changes happening to their software**, and they **do not hold a veto** over the broader migration. Local owners do sometimes question a specific commit, and change authors respond as they would to any other review comment. What earns the arrangement is **a good set of frequently asked questions and a solid historic track record of improvements**.
When humans do the sharding	The rare case where tooling cannot generate the global change, so change generation itself is spread across people.	In early 2017 a vulnerability in a widely used open-source library left any application with a vulnerable version anywhere in its transitive classpath open to remote execution — including, in one case, a municipal transport agency whose systems were encrypted and shut down. Volunteers identified affected projects and sent **more than 2,600 patches**; **more than 50 humans** did the work instead of automated tools.	vs the normal case. This is **much more labour intensive** than automatic generation, and it allows global changes to happen **much more quickly for time-sensitive applications**. Note also what made it possible at all: an existing process, borrowed and pointed outward.

> **CONCEPT BOUNDARY — CENTRALISING THE WORK IS NOT CENTRALISING THE BLAME**
>
> **What it is not:** a claim that consuming teams are careless, or that the infrastructure team should have designed it right the first time. Neither is relevant to who should hold the shovel.
> **What it is:** an argument from three properties, all of which are about efficiency rather than fault. The expertise already exists in one place. The errors fall into a small set of categories, so one team accumulates a playbook. And the team that wants the old system gone is the only party with an incentive to see the last reference removed.
> **The accounting point underneath it:** a migration's cost does not disappear when you decline to fund it — it is simply paid in fragments by hundreds of teams, more slowly, more badly, and often never. Funding a central team looks like adding a cost and is **internalising an externality that already existed**.
> **What it does not buy you:** the right to make changes people do not understand. Owners keep the right to ask what a commit is for; they lose the right to veto the programme it belongs to.

Anchor sketch — where the cost lands

Legend: bold = a named idea **reversed = the failure mode all of this avoids** **Q = cover the page and answer this**

```
THE OBVIOUS PLAN: ship the new system, tell everyone
using the old one to move.
     it seems easier. it does not scale.
                       |
THE UNFUNDED MANDATE  requirements imposed from
outside with no balancing compensation
     benefits DIFFUSE across the org; costs CONCENTRATED
     on each team -> no team ever prioritises it
     +--> and the cost does not vanish. it gets paid
          somewhere, slowly and badly.
                       |
THREE REASONS TO CENTRALISE
-----------------------------------------------------
EXPERTISE IS ALREADY THERE . the team that built the
     system knows its hundreds of thousands of callers.
     making every team relearn that is globally
     inefficient.
     +--> the repeated first change: read the motivation,
          find an example, follow the suggestion, apply
          it locally -- ONCE PER TEAM.
ERRORS REPEAT . they fall into a small set of
     categories, so ONE team accumulates a playbook and
     recovers faster.
INCENTIVES ALIGN . organic migrations do not finish,
     because engineers write new code by copying old
     code -- so the old system keeps producing NEW uses
     of itself. the team that wants it GONE is the team
     that should do the work.
                       |
WHAT THE MACHINERY THEN MAKES POSSIBLE
     FILLING POTHOLES . two header files whose names used
          DIFFERENT SEPARATORS. engineers had to remember
          which -- and found out after a long compile.
          fixed in ~2 weeks of background effort, without
          bothering end users, and build failures from it
          measurably dropped.
     DECISIONS STOP BEING FINAL . the name of a widely used
          symbol; where a popular class lives. this changes
          what you are willing to decide AT DESIGN TIME.
                       |
CATTLE, NOT PETS  a handcrafted change is a pet: days of
work, known intimately, pride on commit. LSC shards are
cattle: nameless, faceless, rolled back at little cost --
unless the whole herd is affected.
     +--> tooling can regenerate them cheaply, so losing
          a few now and then is not a problem.
                       |
THE SOCIAL CONTRACT  owners SHOULD understand what is
happening to their code. owners do NOT hold a veto.
     earned by a good FAQ and a track record.
```

Q Define an unfunded mandate, and say where its cost actually goes.

Q Give the three reasons the work is centralised, and which one is about incentives.

Q What do owners keep, and what do they give up, under the social contract?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Funding a central migration team	Expertise in one place, a playbook for the recurring errors, economies of scale, and an owner who wants it finished.	Visible headcount against a benefit that is spread thinly and hard to attribute.	Whenever the migration matters. The alternative pays the same cost invisibly and more slowly.
Telling each team to migrate itself	No line item, and no central team to staff.	The repeated first change, once per team; migrations that never fully succeed; the old system still producing new uses of itself.	Almost never — and be honest that you have chosen a cost, not avoided one.
Treating shards as cattle	Rejection and rollback become cheap, and tooling can regenerate work at low cost.	The craftsmanship instinct that makes good handcrafted changes has to be switched off.	Any automated migration — and note this is a stance to teach, not a tool to install.
Letting owners question individual commits	Social acceptance, and genuine review comments that catch real problems.	Author time answering questions across many teams.	Always. Answer them as you would any review comment — it is what earns the absence of a veto.
Spending the machinery on small irritations	Measurable reductions in things like build failures, and a productivity and happiness return that outweighs the cost.	Capacity not spent on high-priority migrations.	When the fix is cheap in background effort and the irritation is chronic.
Sharding change generation across humans	Global changes can happen much more quickly when time matters — one campaign sent more than 2,600 patches.	Far more labour intensive than generating changes automatically.	The rare case where tooling cannot generate the change and the deadline is real, such as a live vulnerability.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“We'll announce the new API and let teams move over.”	“That's an unfunded mandate. The benefit is diffuse and the cost is concentrated, so no team will prioritise it.”
“Each team knows their own code best.”	“For their features, yes. For our migration, we hold the context — making 200 teams relearn it is globally inefficient.”
“Adoption is at 60% and stalling.”	“It will. People write new code by copying old code, so the old system keeps generating new uses of itself.”
“Funding a migration team is an extra cost.”	“It's internalising an externality. The cost already exists; right now it's paid in fragments by everyone else.”
“This shard got rejected, I'll work out why.”	“Let it go — it's cattle, not a pet. Regenerate it. Only worry if the whole herd is failing.”
“That team won't approve our change.”	“They can ask what it's for and we answer like any review comment. They don't hold a veto over the programme.”

“That inconsistency is annoying but not worth fixing.”	“With the tooling, that's a couple of weeks of background effort — and we can measure the build failures it removes.”

PART 2 · QUESTION 1

An internal authentication library is being replaced. The platform team has published the new library, written a migration guide, and sent two all-engineering emails. Nine months later, 71% of call sites still use the old library, and the number of *new* call sites on the old library has gone *up* since the guide was published. A vice-president proposes a deadline and a dashboard naming the teams with the most remaining call sites. The platform team's lead argues instead for two engineers funded for six months to do the migration themselves, and is told this is “adding cost to a project that should cost nothing”.

(a) Name what the current approach is, define it, and explain in terms of how benefits and costs are distributed why nine months produced this result. **(b)** Explain the rise in *new* call sites using the stated mechanism, and say what that implies about the migration guide. **(c)** Give three reasons the platform team should hold the work, and say which of the three the dashboard proposal fails to address. **(d)** Answer “adding cost to a project that should cost nothing” directly, in economic terms.

PART 2 · QUESTION 2

A migration team has automated tooling and is landing about 400 shards a week. Three things happen in one week. An engineer on the team spends a day debugging why one particular shard was rejected by a project's checks, and asks to hold the migration until it is understood. A staff engineer on a consuming team objects to a commit in their service and asks the migration be stopped company-wide until a design review is held. And a separate team asks whether the tooling could be pointed at a two-year-old naming inconsistency that causes occasional confusing build errors but that nobody has ever thought worth a project.

(a) Address the rejected shard using the recommended stance toward individual changes, define the two halves of that stance, and give the one condition under which the engineer's instinct would be right. **(b)** Answer the staff engineer: say precisely what they are entitled to and what they are not, and what earns that arrangement. **(c)** Assess the naming request, with reference to the comparable worked example and what made it worth doing. **(d)** Name the wider consequence, for design decisions, of having this capability at all.

2×2 — who is funded, and who wants it finished

COLUMNS: DOES ANYONE HAVE A STAKE IN THE OLD SYSTEM DISAPPEARING? →

	Yes — a team is measured on its removal	**No — everyone benefits a little, nobody a lot**
The work is CENTRALLY funded and staffed	**It finishes** Expertise sits with the people doing the work, the recurring errors accumulate into a playbook, and somebody's job is the last remaining call site. *Move:* stay here, and spend the surplus capacity on the potholes — the chronic small irritations that were never worth a project on their own. This is also the cell where design decisions stop being permanent, which is a benefit you can spend before the migration even starts.	**Funded, and quietly aimless** There is a team and a budget and no owner of the outcome, so work drifts to whatever is tractable and the tail never clears. *Move:* give removal to whoever carries the cost of the old system, measured on the last reference rather than the first thousand.
Left to the teams who use it	**Willing, and each starting from zero** Somebody wants it gone and cannot make it happen, because every consuming team pays the repeated first change: read the motivation, find an example, apply it locally. *Move:* centralise the execution even if the enthusiasm is already distributed. Enthusiasm does not transfer expertise, and 200 teams solving the same problem badly is not a migration.	**The unfunded mandate** Requirements imposed with no balancing compensation, on teams for whom the benefit is too diffuse to matter — while engineers copy existing code and the old system keeps generating new uses of itself. *Move:* stop calling this free. The cost is being paid, in fragments, by everyone; funding a team internalises it and adds economies of scale.

Read it as: the row is where the money is and the column is where the motivation is, and a migration needs both in the same place. Note that the bottom-right cell is the default that arrives when nobody decides anything — it is what an organisation gets for free, and it is the only cell in which the old system outlives everyone who cared about replacing it.

CARRY-OUT RULE FROM THIS PART

Before announcing a migration, name whose budget it lands on. If the answer is “everyone's, a little”, you have not funded a migration — you have written an unfunded mandate, and the honest prediction is that adoption plateaus while the old system keeps acquiring new callers. The question is not whether the cost will be paid; it is whether it will be paid once, by people who know how, or a hundred times by people who don't.

3

PART 3 OF 4

The machine that does it

Four phases, and the two places where the design has to scale sublinearly.

TENTH-GRADER VERSION

- Four steps: get permission, generate the change, break it into pieces and land them, then stop people putting the old thing back.
- Permission isn't bureaucracy. It exists because it became very cheap for one person to create a very large amount of work for other people.
- The generating tool has to take about the same amount of human time no matter how big the codebase is. Otherwise you have just moved the problem.
- Rule of thumb: past about 500 edits, learning the tool beats doing it by hand.
- The pieces are tested in batches on a schedule, like a train leaving every few hours, because testing them one at a time would cost too much.
- One expert reviews all the pieces with pattern-matching tools, and only looks by hand at the odd ones.
- Machine-written changes have to look like human-written ones, or nobody will be able to read the code afterwards.

THE EVERYDAY VERSION

A publisher is changing house style across a back catalogue of thirty thousand titles. Nobody proofreads thirty thousand books. What they do instead is: get the change signed off by a small standards committee, who mostly exist to stop somebody launching a catalogue-wide edit over a comma preference; write one script that applies the rule and prove it on a shelf of samples; then send the books through in batches, running the same set of checks over each batch rather than over each book, because the checks are expensive and most batches pass. One senior editor reviews the output — not by reading the books, but by looking at the *diff patterns* and approving the ones that match what the script was supposed to do, pulling out only the handful that look odd. And afterwards, the style checker is updated so that a new manuscript written in the old style gets flagged at submission. Every part of that is in this section, including the committee's real job, which is not approval but stopping the comma.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
The four phases	**Authorisation**, **change creation**, **shard management**, and **cleanup** — with very nebulous boundaries between them.	These typically happen after a new system, class or function has been written — but it is important to keep them in mind **during the design** of the new system.	The design consequence is explicit: aim to design successor systems **with a migration path from older systems in mind**, so that maintainers can move users automatically. A successor with no migration path has quietly made itself a second system rather than a replacement.

Why authorisation exists	Historically the cost to generate a change and the cost to review it were **somewhat symmetric**, which naturally limited what one person could impose on everyone else.	As tooling improved, it became easy to generate a large number of changes very cheaply — **and equally easy for a single engineer to impose a burden on a large number of reviewers across the company**.	So the process is not there to prohibit these changes but to **keep some oversight and thoughtfulness behind them, rather than indiscriminate tweaking** — the example given is not wanting the tools used to fight over the spelling of a word in comments. Note the underlying economics: the human cost of reviewing far outweighs the technical cost of compute and storage.
The authorisation process	A **lightweight approval process**: a brief document giving the reason for the change, its estimated impact across the codebase — how many smaller shards it would generate — and answers to questions reviewers might have.	Writing it forces authors to think about how to describe the change to an unfamiliar engineer, as a set of frequently asked questions and a proposed change description. Authors also get **domain review from the owners of the interface being refactored**. The proposal goes to an email list of **about a dozen people** with oversight of the whole process.	The committee's goal is **not to prohibit changes but to help authors produce the best possible ones**. Its most common piece of feedback is to **direct all reviews for the change to a single global approver** — first-time authors tend to assume local project owners should review everything. It has historically been **very liberal**, often approving a broad set of related changes rather than one; members can **fast-track obvious changes**; and it is the **escalation body** for conflicts, which in practice has rarely been needed. The only outright rejections have been changes deemed **dangerous**, or **extremely low-value**.
Change creation	Generating the actual edits, **as automated as possible** so that the parent change can be updated as users backslide into old uses or as textual merge conflicts appear.	Backsliding happens for ordinary reasons: copy-and-paste from existing examples, committing changes that have been in development for some time, or simply reliance on old habits.	The scaling requirement is the important part: **human effort must scale sublinearly with the codebase** — it should take roughly the same amount of human time to generate the collection of all required changes **no matter the size of the repository**. The tooling must also be **comprehensive**, so an author can be assured the change covers every case they are trying to fix.
The 500-edit rule of thumb	If a change requires **more than 500 edits**, it is usually more efficient for an engineer to learn and execute the change-generation tools than to make the edits manually.	For experienced “code janitors”, that number is often **much smaller**.	vs a threshold for when a change counts as large. It is a threshold for **when to stop doing it by hand** — and, like the rest, an early investment in tooling usually pays off in the short to medium term.
Human-looking output	Whatever generates the changes, the result should **look as much like a human-generated change as possible**, because the codebase is optimised for human readability.	Every generation tool runs the automatic formatter for the language being changed **as a separate pass**, so the change-specific tooling does not have to concern itself with formatting.	This is stated as load-bearing rather than cosmetic: **without automated formatting, large-scale automated changes would never have become the accepted status quo.** Automatic formatters are described as a crucial part of the infrastructure, and the requirement leads directly to the need for style guides.

Sharding	Taking the large change and splitting it **based on project boundaries and ownership rules** into changes that can be submitted atomically, then putting each through an **independent test-mail-submit pipeline**.	Because the sharding platform is a heavy user of shared developer infrastructure, it **caps the number of outstanding shards** for any given change, **runs at lower priority**, and **communicates with the rest of the infrastructure** about how much load is acceptable to generate.	The requirement on shards: they must be **committable independently** — either having no interdependence, or with the sharding mechanism grouping dependent changes together, such as a header file and its implementation. They must also pass project-specific checks like any other change.
The train model	Testing many changes together rather than one at a time. It exploits two facts: these changes **rarely interact with one another**, and **most affected tests pass for most of them**.	It relies on the changes tending to be **pure refactorings, very narrow in scope, preserving local semantics**, and on individual changes being **simpler and highly scrutinised, so correct more often than not**.	Its advantage over isolated runs: it works for multiple changes at once and does not require each to ride alone. An isolated run can be requested, but these are **very expensive and performed only during off-peak hours**.
The five steps of the train	Started fresh **every three hours**.	**1.** For each change on the train, run a sample of **1,000 randomly selected tests**. **2.** Gather all the changes that passed and create one uber-change from them — the train. **3.** Run the union of all tests directly affected by the group, which for a large or low-level change can mean **every single test in the repository**, and **can take more than six hours**.	**4.** For each *non-flaky* test that fails, rerun it individually against each change on the train to determine which one caused it. **5.** Generate a report per change describing all passing and failing targets, which **can be used as evidence that the change is safe to submit**.
Testing the shards	Each shard is run over the **transitive closure of every test it might affect** rather than the standard project-based presubmit tests, because a shard can contain arbitrary files.	In practice the **vast majority of shards affect fewer than a thousand tests**, out of the millions across the codebase. For those affecting more, shards are grouped: first run the union of all affected tests for all shards, then for each shard run only the intersection of its affected tests with those that failed the first run. Because most of these unions cause almost every test to run, **adding more changes to that batch is nearly free**.	The drawback of running so many tests: **independent low-probability events become near-certainties at large enough scale.** Flaky and brittle tests do little harm to the teams that own them and seriously affect the throughput of a migration system. Confidence in machine-generated, semantics-preserving changes is now high enough that **recently flaky tests are ignored when submitting via the automated tooling** — in theory a regression could then be caught only by a flaky test turning into a failing one, and in practice this is rare enough to handle by human communication rather than automation.
Mailing reviewers	Once a shard is validated by testing, it is mailed to an appropriate reviewer — and in a company with thousands of engineers, **reviewer discovery is itself a challenging problem**.	Ownership files list who may approve changes to a subtree; an owners-detection service reads them and **weights each owner by their expected ability to review the specific shard**. If an owner proves unresponsive, **additional reviewers are added automatically**.	Two checks are deliberately relaxed. Per-project precommit checks run, but **certain checks are selectively disabled** — such as those for non-standard change description formatting, which are useful locally and a source of friction at scale. And **preexisting check failures are aggressively ignored**: local code owners are responsible for having none, **as part of the social contract** between them and the infrastructure teams.

The global approver	Someone with ownership rights to approve any change throughout the repository, to whom all shards of a migration are assigned instead of the owners of individual projects.	Global approvers generally have specific knowledge of the language or libraries involved, work with the change author to know what to expect, and **know the potential failure modes**, so they can customise their workflow.	The mechanism that makes it scale: rather than reviewing each change individually, they use **a separate set of pattern-based tooling** to review and **automatically approve the ones that meet their expectations**, manually examining only the small subset that are anomalous through merge conflicts or tooling malfunction. Local owners are sent changes **only where their review is required for context, not just for approval permissions** — because in practice they trust the generating engineers too much and give these changes only cursory review.
Cleanup	Different migrations have different definitions of "done" — from completely removing an old system, to migrating only high-value references and leaving the rest to disappear organically.	In almost all cases it is important to have **a system that prevents additional introductions** of the symbol or system the change worked to remove.	The mechanism is review-time: a static-analysis framework flags when an engineer introduces a new use of a deprecated object, which **has proven an effective method to prevent backsliding**. Note the honest caveat about the organic option: the systems you most want to decompose organically are the ones most resistant to doing so.

CONCEPT BOUNDARY — TWO THINGS MUST SCALE SUBLINEARLY, AND THEY ARE DIFFERENT THINGS

Generation: the human effort to produce the complete set of edits must be roughly constant in the size of the repository. If writing the change takes twice as long in a codebase twice the size, you have automated the typing and not the problem.

Review: the human effort to approve the shards must not grow with the number of shards. This is what the global approver plus pattern-based tooling buys — approve what matches expectation automatically, examine only the anomalies.

Why both are required: they fail independently and each failure is fatal on its own. Perfect generation with per-project review means one engineer can impose unbounded work on hundreds of reviewers, which is the exact asymmetry that made an approval process necessary. Perfect review with hand-written generation means the change never gets made in the first place.

The thing that is allowed to scale linearly: compute. Running the same tests repeatedly, and running every test in the repository for a low-level change, are accepted costs — because engineer time is the expensive resource and machine time is not.

Anchor sketch — the four phases, end to end

Legend: bold = a named phase or component **reversed = the property the whole design exists to preserve** **Q = cover the page and answer this**

1. AUTHORISATION
WHY it exists: generating changes got cheap; REVIEWING them did not, so one engineer can now burden hundreds of reviewers.
a brief doc: reason . estimated impact (how many shards) . answers to likely questions . an FAQ . + domain review from the interface's owners -> an email list of ~a dozen overseers
most common feedback: send ALL reviews to ONE GLOBAL APPROVER (first-timers assume local owners should review everything)
VERY LIBERAL . often approves a broad family at once . obvious ones fast-tracked . also the ESCALATION body, rarely needed . rejects only the DANGEROUS or the EXTREMELY LOW VALUE
|
2. CHANGE CREATION
HUMAN EFFORT MUST SCALE SUBLINEARLY -- roughly the same human time whatever the repository's size, and COMPREHENSIVE: every case you meant to fix
keep regenerating: users BACKSLIDE (copy-paste, long-lived branches, old habits) and files conflict
>500 edits? learn the tool ("code janitors": far less)
run the language's FORMATTER as a separate pass, so machine output reads like human output
|
3. SHARD MANAGEMENT split by project boundary and ownership into ATOMICALLY submittable pieces, each through its own TEST -> MAIL -> SUBMIT pipeline
caps outstanding shards . LOWER priority . negotiates load with shared infrastructure
TEST . transitive closure of every test it may hit. THE TRAIN: fresh every 3 h .. 1) 1,000 random tests per change 2) merge the passers into one uber-change 3) run the union -- can be EVERY test, 6+ hours 4) rerun each NON-FLAKY failure against each change 5) report = evidence it is safe to submit. most shards touch <1,000 tests of millions. flaky tests IGNORED on auto-submit
MAIL . owners-detection weights owners by expected ability; unresponsive -> add more. disable formatting-style checks; IGNORE failures that PREDATE the change (the owners' half of the deal)
REVIEW . one GLOBAL APPROVER + pattern-based tooling auto-approves what matches expectation; a human sees only anomalies. local owners only where review gives CONTEXT, not approval rights
|
4. CLEANUP "done" varies: full removal .. or migrate the high-value references and let the rest fade
ALWAYS: stop new introductions -- flag new uses of the deprecated thing AT REVIEW TIME.

Q Give the four phases, and say what the first one is actually protecting.

Q Give the five steps of the train, with the two numbers attached to them.

Q How does review avoid growing with the number of shards?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
An approval committee for large changes	Oversight and thoughtfulness instead of indiscriminate tweaking, plus an escalation path for conflicts.	A step before the work, and a body that could become a bottleneck.	Once generating changes is cheap and reviewing them is not — keep it lightweight and liberal.
Routing all reviews to a global approver	Review stops growing with the number of shards; one expert knows the failure modes and can automate approval.	Local owners see less of what lands in their code.	For mechanical changes. Send to local owners only where their review supplies context.
Batching changes onto a test train	Far fewer total test executions, and a per-change report that serves as evidence.	A failure has to be attributed by rerunning against each change on the train.	When changes rarely interact and most tests pass for most of them — which is what makes the model work at all.
Ignoring recently flaky tests on automated submit	Throughput, in a regime where low-probability failures are near-certainties at scale.	In theory, a regression detectable only by a flaky test turning into a failing one.	When confidence in machine-generated, semantics-preserving changes exceeds confidence in the flaky test — and handle the rare miss by talking to people.
Running the formatter as a separate pass	Machine output that reads like human code, and generation tooling that never has to think about formatting.	An extra pass, and a house style everyone must accept.	Always — without it, automated change at this scale does not become socially acceptable.
Ignoring preexisting check failures	The migration is not blocked by problems it did not cause.	It relies on owners keeping their own house in order.	Where that expectation is explicit — it is one half of the social contract, not a unilateral decision.
Preventing reintroduction at review time	The migration's gains are not eaten by new uses appearing behind it.	Another warning in reviewers' faces, which has its own budget.	Almost always — without it, cleanup never converges.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“Why do we need approval to fix our own code?”	“Because generating changes got cheap and reviewing them didn't. The committee is protecting other people's attention.”
“I'll script it, then hand-edit the leftovers.”	“Then the tool isn't comprehensive. Generation has to cover every case and take the same human time at any repo size.”
“It's only about 800 edits, I'll just do them.”	“Past roughly 500, learning the tool wins — and you'll need it again when people backslide.”

“We'll ask each project's owners to review their shard.”	“That's the mistake first-time authors make. Route it to a global approver; send locals only what needs their context.”
“We should test each shard in isolation.”	“Isolated runs are very expensive and off-peak only. Put them on the train and attribute failures afterwards.”
“A flaky test is blocking the migration.”	“At this scale, flakes are certainties. We trust a semantics-preserving generated change more than a recently flaky test.”
“Their presubmit was already red before we got there.”	“Then we ignore it. Keeping their own checks green is their half of the contract.”
“The migration's at 100%, we're done.”	“Not until new uses can't be added. Cleanup means flagging reintroductions at review time.”

PART 3 · QUESTION 1

An engineer proposes replacing a deprecated date-formatting call across an estimated 12,000 sites. Their plan: write a script over a weekend, generate all the edits in one pass, and mail each resulting shard to the owners of the project it touches. In the first week, 340 shards go out; by the following Monday, 190 are unreviewed, four project owners have complained about the volume, nine shards have failed on checks that were already failing before the migration touched them, and roughly 60 sites the script did not recognise have been found by hand. Two new uses of the deprecated call have appeared in code written that week.

(a) Name the four phases and say which one this plan skipped entirely, giving the specific reason that phase exists. **(b)** The review backlog is a design failure, not a staffing one. Name the property review must have, the mechanism that provides it, and how that mechanism actually processes 340 shards. **(c)** Address the nine already-failing checks and say whose responsibility they are and under what arrangement. **(d)** The 60 unrecognised sites and the two new uses are different failures. Name the property each one violates, and the phase that answers each.

PART 3 · QUESTION 2

A team is preparing a migration that will produce about 2,000 shards. They intend to run each shard's full transitive test set individually, on the shared CI cluster, at normal priority, as soon as each shard is generated. They have also decided to block submission of any shard whose test run shows a failure of any kind, on the grounds that this is the safe choice. Finally, their generation tool produces correct but unformatted output, which they plan to tidy up in review comments.

(a) Describe the batching model they should use instead: the two facts it exploits, its five steps, and the two numbers attached. **(b)** Give the grouping technique for shards affecting large numbers of tests, and why adding more to such a batch is nearly free. **(c)** Their blocking rule will stall: explain what happens to low-probability failures at this scale, the position taken on recently flaky tests, and the residual risk. **(d)** Say what is wrong with tidying formatting in review, including the claim about what would not have happened without automated formatting.

2×2 — the two costs that must not grow with you

COLUMNS: DOES REVIEW EFFORT GROW WITH THE NUMBER OF SHARDS? →

	No — one approver, pattern-based auto-approval	**Yes — every shard read by its project's owners**
Generation effort is FLAT in the size of the repository	**The machine works** Roughly constant human time to produce every edit, and review that examines only anomalies. The only cell where one process serves a migration of twelve files and one of twelve thousand. *Move:* keep both halves honest as you grow — regenerate rather than patch, and check the approver's tooling still recognises what the generator emits.	**One engineer, unbounded work for everyone else** Cheap to create and expensive to review is exactly the asymmetry that made an approval process necessary. *Move:* route shards to a global approver, and send local owners only what needs their context rather than their permission. Otherwise expect cursory review — they trust the generating team.
Generation effort GROWS with the codebase	**Automated typing, not automated change** Review scales fine; the change is a hand-fed pipeline that slows as the repository grows and will not be comprehensive. *Move:* past roughly 500 edits, build the tool — you will need it again anyway, to regenerate the parent change as people backslide.	**A migration that will not finish** Both costs grow, so the effort ends when the enthusiasm does, and the tail — awkward projects, unrecognised call sites — never gets done. *Move:* do not start here. The committee's most common advice fixes the column; the 500-edit heuristic fixes the row.

Read it as: both axes are about human time, and the one resource deliberately allowed to grow — compute — appears on neither. Running the same tests repeatedly and running every test in the repository for a low-level change are accepted prices, because an engineer hunting a failure costs more than a machine repeating one.

CARRY-OUT RULE FROM THIS PART

Ask of any migration plan: what happens to it if the codebase doubles? If generating the change takes twice as long, you automated the typing. If reviewing it takes twice as long, you have moved your problem onto people who did not choose it. Both answers should be “roughly the same”, and only compute should be allowed to double.

4

PART 4 OF 4

Deciding what to remove

Code as a liability, the two kinds of deprecation, and why removal is the hardest change there is.

TENTH-GRADER VERSION

- Code isn't treasure. It's a bill that keeps arriving. The valuable thing is what the code *does*, not the code.
- Old doesn't mean obsolete. Something can be forty years old and still exactly right.
- Removing a system is the biggest change you can make — you're not altering the behaviour, you're deleting it — so everyone who was quietly depending on it finds out at once.
- People also just don't want to. They built it, they like it, and nobody gets promoted for deleting things.
- Two kinds. Advisory: “this is old, please move.” Compulsory: “this stops working on a date.”
- Advisory alone almost never finishes, because the old system keeps attracting new users. Hope is not a strategy.
- Compulsory without staff is just bullying. If you mean it, fund the team that does the moving.
- And a warning is only useful if the person can act on it, at the moment they can act on it.

THE EVERYDAY VERSION

A hospital wants to retire an old drug protocol that a better one has replaced. Notice how little of this is medical. The new protocol is better but it is not identical, so every existing patient has to be looked at individually rather than switched by a rule. Some wards have been using the old one in ways nobody documented — off-label, in combination, at doses the original authors never anticipated — and you only discover those when you try to take it away. The consultant who wrote the old protocol is still on staff and is not thrilled. Meanwhile the cost of leaving it in place is real but invisible: two sets of training, two sets of stock, two sets of interactions to check. And there is a limit to how much of this you can run at once — if you retire every protocol in the hospital simultaneously, nobody can treat anyone. So you pick a few, you staff them properly, and you set a date that you actually mean.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Deprecation	The process of **orderly migration away from and eventual removal of obsolete systems**.	It belongs to software engineering rather than programming, because it requires managing a system over time. Done correctly it **reduces resource costs and improves velocity** by removing redundancy and complexity that build up.	vs an unambiguous good. **Poorly deprecated systems may cost more than leaving them alone.** They range from a function call to an entire stack, and can be planned for *during design*.

Code is a liability, not an asset	The premise the whole discussion begins from. If code were an asset, there would be no reason to spend time removing it.	Code has costs borne at creation and **many more borne across its lifetime** — the resources to keep it running, and the effort to update it as the ecosystem around it evolves.	The resolution of the apparent paradox: **code itself doesn't bring value — the functionality it provides does**, and that functionality is an asset if it meets a user need. Given the same functionality from one line of maintainable code or ten thousand of convoluted code, prefer the former. So the metric to maximise is **functionality delivered per unit of code**, and one of the easiest ways to improve it is removing what is no longer needed.
Age is not obsolescence	**The age of a system alone doesn't justify its deprecation.**	A system may be finely crafted over years into the epitome of software form and function; some have been improved over decades, with changes now few and far between.	What does justify it: deprecation **is best suited for systems that are demonstrably obsolete and for which a replacement exists that provides comparable functionality** — one that uses resources more efficiently, has better security properties, is built more sustainably, or just fixes bugs. **Just because something is old, it does not follow that it is obsolete.**
The cost of keeping both	Having two systems doing the same thing may not seem pressing, and **over time the costs of maintaining both can grow substantially**.	Users may need the new system while still depending on the old. The two may need to interface, **requiring complicated transformation code**, and as both evolve they can come to depend on each other.	The cost people miss: having multiple systems performing the same function **impedes the evolution of the newer system**, because it is still expected to maintain compatibility with the old one. So removing the old system pays off twice — the maintenance saved, and the replacement now free to evolve faster.
The simultaneity limit	Organisations have **a limit on the amount of deprecation work that is reasonable to undergo simultaneously** — both for the teams doing it and for their customers.	Everybody appreciates freshly paved roads; close every road at once and nobody goes anywhere. By focusing effort, crews finish specific jobs faster **while also allowing other traffic to make progress**.	The instruction that follows is about commitment, not just sequencing: **choose deprecation projects with care and then commit to following through on finishing them.**
Why removal is the hardest change	Hyrum's Law applies with full force: **the more users of a system, the higher the probability that users are using it in unexpected and unforeseen ways**, and the harder it is to deprecate and remove.	Their usage just **happens to work** instead of being **guaranteed to work**. Removing a system can be thought of as **the ultimate change**: not merely changing behaviour but removing it completely — an alteration radical enough to **shake loose a number of unexpected dependents**.	Two further difficulties compound it. Deprecation usually isn't even an option until a newer system provides the same or better functionality — and that new system **is also different**, since an identical one would offer users no benefit, so a one-to-one match is rare and **every use of the old system must be evaluated in the context of the new**. And there is emotional attachment: resistance of the form **“I like this code!”**, which is understandable and ultimately self-defeating.

The political difficulty	Funding and executing removal costs real money, **whereas the costs of doing nothing and letting the system lumber along unattended are not readily observable**.	It is hard to convince stakeholders it is worthwhile, especially where it slows new feature development. Research techniques can supply concrete evidence that it is.	One counter to the emotional objection is infrastructural: making the source repository searchable **not just at trunk but historically**, so that **even code that has been removed can be found again**. There is also an in-house joke worth knowing, because it names a real condition: there are two ways of doing things — **the one that's deprecated, and the one that's not-yet-ready**.
Incrementality	Given the difficulty, it is often easier to **evolve a system in situ rather than completely replacing it**.	Migrating to entirely new systems is **extremely expensive, and the costs are frequently underestimated**. In-place refactoring keeps existing systems running while still delivering value.	vs an escape from the process. **Incrementality doesn't avoid the deprecation process altogether** — it breaks it into smaller, more manageable chunks that yield incremental benefits.
Design for deprecation	Planning a system's eventual removal while it is first being built — radical in software, **common in other engineering disciplines**.	A nuclear power station's decommissioning is taken into account as part of its design, **even to the point of allocating funds for it**, and many design choices follow from knowing it will be needed.	Two questions to ask of a new system: **how easy will it be for my consumers to migrate from my product to a potential replacement?** and **how can I replace parts of my system incrementally?** Why it rarely happens: engineers are drawn to building rather than maintaining, cultures emphasise shipping quickly, and it is psychologically hard to plan the demise of what you are building. The blunt version: **don't start projects your organisation isn't committed to support for its own expected lifespan.**
Advisory deprecation	Deprecations that **don't have a deadline and aren't high priority** for the organisation, and for which it isn't willing to dedicate resources. Also called **aspirational**.	The team knows the system has been replaced and hopes clients will eventually migrate, without imminent plans either to help them move or to delete the old system. **This kind of deprecation often lacks enforcement** — and as the production engineers will readily tell you, **“hope is not a strategy.”**	What it is good for: advertising a new system's existence and encouraging early adopters. Two conditions attach. The new system **should not be considered in a beta period** — it must be ready for production use and loads and prepared to support new users indefinitely. And the benefits **cannot be simply incremental: they must be transformative**, because users will not migrate on their own for marginal gains and **even systems with vast improvements will not gain full adoption from advisory efforts alone**. Existing uses exert a **conceptual pull**: many uses of the old system tend to pick up a large share of new uses, no matter how much you say “please use the new system”.

Compulsory deprecation	Active encouragement, **usually with a deadline for removal**: if users continue to depend on the old system beyond that date, **they will find their own systems no longer work**.	Counterintuitively, the way it scales is **by localising the expertise of migrating users within a single team of experts** — usually the team responsible for removing the old system entirely, which has the incentive to help others move and develops experience and tools the whole organisation can use.	Two requirements and one hazard. It **needs an enforcement mechanism** — not that the schedule can never change, but that the team is empowered to break non-compliant users after sufficient warning; without it, customer teams deprioritise the work in favour of features. And it **must be actively staffed by a specialised team through completion**, or it reads as mean spirited and as exactly the unfunded mandate it is. The hazard is political: if the last remaining user is critical infrastructure, **it is hard to believe the deprecation is really compulsory if that team can veto its progress**.
Finding hidden dependents	Some teams **might not even know they have a dependency** on an obsolete system, and these can be difficult to discover analytically.	They can be found dynamically, through **tests of increasing frequency and duration during which the old system is turned off temporarily** — planned outages announced in the months and weeks beforehand.	These intentional changes discover unintended dependencies **by seeing what breaks**, alerting teams to prepare for the deadline or to work with the deprecating team to adjust the timeline. A related trick for static dependencies: **changing the name of implementation-only symbols to see which users were depending on them unaware**. For static code dependencies, semantic analysis is often sufficient to find everything without this.
Warnings: actionable and relevant	Any deprecation warning issued to a user needs **two properties**. **Actionable**: the user can use it to perform some relevant action — not just in theory but in practical terms, given the expertise expected of an average engineer. **Relevant**: it surfaces **at a time when the user actually performs the indicated action**.	Actionable: a tool warns that a call should be replaced with its updated counterpart, or an email outlines the steps to move data. Relevant: warn about a deprecated function **while the engineer is writing the code that uses it**, not weeks after it was checked in; send a data-migration email **several months before** removal, not the weekend before.	The failure mode is accumulation. In a transitive context — a library depending on a library depending on one that issues the warning — warnings **can soon overwhelm users to the point where they ignore them altogether**, which in health care is called **alert fatigue**. Hence: mark liberally but surface in targeted ways, such as limiting warnings **to newly changed lines**, and add intrusive ones **only for compulsory deprecations where the team is actively moving users**.
Process owners and milestones	**Without explicit owners, a deprecation process is unlikely to make meaningful progress**, no matter how many warnings and alerts the system generates.	The alternatives are worse: delegating to users **becomes simply an advisory deprecation, which will never organically finish**, and deprecating nothing is **a commitment to maintain every old system indefinitely**. Such teams need **removal as their primary goal, not a side project** — in a priority contest, deprecation always loses.	On milestones: it can feel that the only one is removing the system entirely — and if the team has done its job that step is **the least noticed by anyone external**, because by then it has no users. Resist making it the only measurable milestone; **it might not even happen in all deprecation projects**. Use incremental milestones that are measurable and deliver value, recognising things like **deleting a key subcomponent** as you would a new product.

> **CONCEPT BOUNDARY — ADVISORY AND COMPULSORY ARE NOT TWO INTENSITIES OF THE SAME THING**
>
> **What they look like:** points on a dial, where compulsory is advisory with more pressure applied.
> **What actually separates them:** a deadline with enforcement behind it, and a funded team doing the migrating. Remove either and you get a different failure mode, not a milder deprecation — a deadline with no staffing is an unfunded mandate that reads as mean spirited; staffing with no enforcement is a team pushing on customers who can always choose features instead.
> **What advisory is genuinely for:** advertising and early adopters, on two conditions — a production-ready new system, and *transformative* rather than incremental benefits. As a strategy for finishing it fails, because existing uses pull new uses toward the old system whatever the documentation says.
> **The honest test:** if the last remaining user could refuse and survive, it is advisory whatever you called it.

Anchor sketch — deciding, and then finishing

Legend: bold = a named idea **reversed = the premise the rest depends on** **Q = cover the page and answer this**

```
CODE IS A LIABILITY, NOT AN ASSET
     the FUNCTIONALITY is the asset; code is a means. same
     behaviour in 1 line or 10,000? prefer 1. maximise
     FUNCTIONALITY PER UNIT OF CODE -- and the easiest
     lever is removing what is not needed.
                       |
BUT AGE IS NOT OBSOLESCENCE. deprecate what is
DEMONSTRABLY OBSOLETE and has a replacement with
COMPARABLE functionality.
     keeping both costs: transformation code . mutual
     dependence . the OLD holds back the NEW, which must
     stay compatible. and a LIMIT on how many at once.
                       |
WHY REMOVAL IS THE HARDEST CHANGE
     HYRUM'S LAW . the more users, the more depend on
          behaviour you never promised: it "happens to
          work", not "guaranteed to work"
     THE ULTIMATE CHANGE: deleting behaviour, not altering
          it. unknown dependents fall out.
     the replacement is BETTER AND DIFFERENT -> 1:1 rare
     "I LIKE THIS CODE!" . understandable, self-defeating
     POLITICS . removal costs real money; doing nothing
          costs money you cannot see
     +--> cheaper: evolve IN PLACE. chunks it, not avoids.
                       |
DESIGN FOR IT UP FRONT (a nuclear plant allocates
DECOMMISSIONING FUNDS at design time)
     how easily can consumers migrate OFF me?
     how can parts of me be replaced INCREMENTALLY?
     do not start what you will not support.
                       |
TWO KINDS
-------------------------------------------------------
ADVISORY . no deadline, no priority, no resources.
     aspirational. "HOPE IS NOT A STRATEGY."
     good for advertising and early adopters. needs a
          PRODUCTION-READY new system and TRANSFORMATIVE,
```

```
            not incremental, benefits.
      fails because EXISTING USES PULL: the old system
            keeps taking a big share of NEW uses
COMPULSORY . a DEADLINE. miss it and your system
      stops working.
      scales by LOCALISING migration expertise in ONE team
      needs ENFORCEMENT (power to break non-compliant users
            after fair warning) AND STAFFING, or it is an
            unfunded mandate that reads as mean spirited
      hazard: if the last user is critical infra and can
            veto, it was never compulsory
      find hidden dependents: planned outages of INCREASING
            DURATION; rename internal-only symbols
                        |
WARNINGS need BOTH properties, or they are noise
      ACTIONABLE . the reader can do the next step
      RELEVANT ... it appears WHEN they are doing the thing
      too many -> ALERT FATIGUE (a term from health care)
      so mark liberally, SURFACE NARROWLY: newly changed
      lines only; intrusive ones only for COMPULSORY,
      actively staffed deprecations.
                        |
OWNERS AND MILESTONES
      no explicit owner -> no progress, however many
            warnings the system emits
      alternatives are worse: delegate to users = advisory,
            never finishes; do nothing = maintain forever
      removal must be the PRIMARY GOAL, not a side project
      celebrate INCREMENTAL milestones. the final deletion
            is the least noticed thing you will ever do.
```

Q Why is code a liability, and what is the asset?

Q Give the two properties a warning needs, and what happens without them.

Q What does compulsory deprecation need that advisory does not, and what happens if you skip each?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Removing an obsolete system	Maintenance saved, and a replacement that can finally evolve without staying compatible with its predecessor.	Real, visible money — against a cost of inaction that nobody can see.	When it is demonstrably obsolete and a replacement offers comparable functionality. Age alone is not a reason.
Evolving in place instead of replacing	Usually much cheaper; existing systems keep running while value is still delivered.	You still have to deprecate, just in chunks.	Most of the time — wholesale migration is extremely expensive and the cost is frequently underestimated.

Advisory deprecation	Advertises the new system and picks up early adopters, cheaply.	It will not finish. Existing uses pull new uses toward the old system regardless of what you announce.	When the new system is production-ready and its benefits are transformative — and you are not relying on it to complete.
Compulsory deprecation	It actually ends, and the expertise concentrates in one team that gets better at it.	A funded team, a defensible deadline, and the political capital to break someone.	When the removal matters enough to staff. Without staffing it is an unfunded mandate wearing a deadline.
Planned outages of increasing duration	Discovers dependents nobody knew about, early enough for them to prepare or negotiate.	Deliberately breaking things, on purpose, on a schedule.	When dependencies are dynamic and cannot be found by static analysis.
Marking many things deprecated	Cheap, and honest about what is on the way out.	Alert fatigue — warnings accumulate transitively until they are ignored altogether.	Fine, provided you surface them narrowly — on newly changed lines, and intrusively only where a staffed compulsory effort is under way.
Staffing a dedicated removal team	The only arrangement that reliably finishes, since deprecation loses every priority contest it is put in.	Headcount on deconstruction rather than construction.	Whenever you have decided the system must go. Removal has to be their primary goal, not a side project.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“We've built up a lot of valuable code.”	“We've built up a lot of liability. The functionality is the asset — what's our functionality per unit of code?”
“That system is ancient, we should replace it.”	“Old isn't obsolete. Is there a replacement with comparable functionality, and what is the old one actually costing us?”
“Nobody should still be using that.”	“They are, in ways we never promised. Removal is the ultimate change — expect dependents we've never heard of.”
“We'll mark it deprecated and see what happens.”	“That's advisory, and hope is not a strategy. It'll reduce new uses slightly and migrate almost nobody.”
“We've set a deadline, so it's compulsory.”	“Only if we can enforce it and we're staffing the migration. Otherwise it's an unfunded mandate with a date on it.”
“Everyone's ignoring the deprecation warnings.”	“Alert fatigue. Are they actionable, and are they surfacing while people are writing the code — or weeks later?”
“We can't find who depends on it.”	“Turn it off for five minutes, then an hour, then a day. What breaks is your dependency list.”
“The team has nothing to show until it's deleted.”	“Then pick milestones that land sooner. The final deletion is the least visible thing they'll ever do — by then there are no users.”

PART 4 · QUESTION 1

A team owns a message-serialisation library that a newer one has replaced. Eighteen months ago they added a compile-time warning to every public function in the old library, wrote a migration page, and announced it. Today: the warning fires roughly 40,000 times per build in downstream projects, transitively; adoption of the new library is 22%; the number of call sites on the old library has grown by 4%; and one team told them, politely, that they filter the warnings out of their build logs. The new library is faster to serialise by about 8% and otherwise equivalent. The team's plan is to add a second, louder warning.

(a) Name the kind of deprecation this is, define it, and give the phrase used for the strategy it amounts to. **(b)** The 4% growth is predicted by a named mechanism. State it, and say what it implies about the migration page. **(c)** Assess the warnings against both properties they must have, name the condition the 40,000 figure has produced, and give two specific ways to surface warnings without it. **(d)** Advisory deprecation has a stated precondition about the *benefits* of the new system. State it, apply it to the 8% figure, and say what the team must change if they want this to finish.

PART 4 · QUESTION 2

An organisation decides to remove an old job-scheduling service by the end of the year. It announces the date, publishes a migration guide, and asks each consuming team to move itself. Nine months in, three things are true. Most teams have moved. The remaining users include the company's revenue-reporting pipeline, whose lead has said publicly that they will not be ready and that switching them off is “not going to happen”. And a second, unrelated turndown was started in parallel two months ago, so the platform group is now running both while also on call for the old service. A director suggests that since the deadline clearly cannot hold, they should drop it and let the remaining users move when convenient.

(a) Name the kind of deprecation intended and the two things it requires; say which one the organisation supplied and which it did not. **(b)** The reporting pipeline is the textbook hazard. State it in general terms and say what its existence proves about the deprecation's true category. **(c)** Assess running two turndowns at once, using the stated limit and the analogy given for it. **(d)** Evaluate the director's suggestion: name exactly what the deprecation becomes, what that predicts, and the alternative — including what should happen to the team's milestones.

2×2 — the deadline, and who does the moving

COLUMNS: IS A DEDICATED TEAM DOING THE MIGRATION WORK? →

	Yes — staffed through to completion	No — users move themselves
COMPULSORY — a deadline with real enforcement	**It ends** Expertise concentrates in the team removing the old system, which has both the incentive and the tools; the deadline gives customer teams a reason to accept help rather than defer it. *Move:* keep the enforcement credible, because the whole cell rests on it — if the last remaining user can refuse and survive, you are in the row below whatever the announcement said. Find the unknown dependents early with outages of increasing duration, and celebrate incremental milestones, since the final deletion will be invisible.	**An unfunded mandate with a date on it** Customers see work pushed onto them to keep their own services running, which is exactly the friction that makes them deprioritise it — and it comes across as mean spirited. *Move:* either staff it or stop calling it compulsory. A deadline is not a substitute for the migration expertise the removing team already has and every consuming team would have to acquire separately.
ADVISORY — no deadline, no dedicated resources	**Generous, and unfinished** Real help is available and there is no reason to take it this quarter, so the tail persists and the team's own removal date keeps moving. *Move:* this is a reasonable place to start and a bad place to stay. Use it to advertise and to gather early adopters, then convert to a deadline once the new system has proved itself in production.	**Hope is not a strategy** The honest description of marking something deprecated and walking away. It produces slightly fewer new uses and rarely leads to anyone actively migrating. *Move:* expect the conceptual pull — the old system will keep taking a large share of new uses however often you say otherwise. If the benefits are genuinely transformative this can still work; if they are incremental, nothing here will finish.

Read it as: the row is what you have announced and the column is what you have paid for, and only the top-left is a deprecation that ends. Note that the two off-diagonal cells fail in opposite directions — the top-right pushes cost onto people who did not choose it, the bottom-left offers help nobody has a reason to accept — and each is fixed by the other one's strength.

CARRY-OUT RULE FROM THIS PART

Before announcing a deprecation, ask what happens to the last user who refuses. If the answer is “their system stops working, and we have engineers to help them avoid that”, it is compulsory and it will finish. If the answer is “we'd have to make an exception”, it is advisory — and the right response is not a louder warning but either the staffing to make it real or the honesty to call it what it is.

SECTION 6

Concept sketch — the whole subject, end to end

Four named groups, in the order the story runs: **THE CONSTRAINT → THE ECONOMICS → THE MACHINE → THE DECISION**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: bold = a group heading **reversed = the one group to remember** **Q = retrieval cue**

The four groups run in order: **THE CONSTRAINT** — why atomic fails · **THE ECONOMICS** — who pays · **THE MACHINE** — how it is done · **THE DECISION** — what to remove.

GROUP 1 - THE CONSTRAINT . the atomic commit shrinks

AN LSC is logically related changes that CANNOT PRACTICALLY be submitted as one atomic unit. not "big" -- "not atomic". scales down to a few dozen changes.
Q what makes something an LSC, if not its size?

AS THE CODEBASE AND HEADCOUNT GROW, THE LARGEST ATOMIC CHANGE GETS SMALLER.

FIVE BARRIERS VCS limits (linear ops, blocked writers) . merge conflicts (files x engineers) . haunted graveyards . heterogeneity . testing
Q which barrier does testing solve, and which does it worsen?
Q 1 bad file in 25 vs in 10,000 -- what is the point?

GROUP 2 - THE ECONOMICS . whose budget does it land on

THE UNFUNDED MANDATE benefits diffuse, costs concentrated -> nobody prioritises it. the cost does NOT vanish.
Q where is that cost actually paid?

CENTRALISE, for three reasons: the expertise is already there . errors repeat, so one playbook . and only the team that wants it GONE has the incentive to finish
Q why do organic migrations stall, mechanically?

THE PAYOFFS potholes get filled . decisions stop being final . shards are CATTLE, not pets
Q what does "cattle not pets" license you to do?

THE CONTRACT owners understand; owners do not veto
Q what earns that arrangement?

GROUP 3 - THE MACHINE . four phases, two sublinear costs

---

1 AUTHORISE a brief doc + FAQ + domain review -> ~a dozen overseers. exists because generating got cheap and REVIEWING did not. liberal; fast-tracks; escalation.
Q what does the committee most commonly tell you to do?

2 GENERATE human effort FLAT in repo size, and COMPREHENSIVE. >500 edits -> use the tool. run the formatter so machine output reads as human.
Q why must you keep regenerating the parent change?

3 SHARD split by project + ownership -> independent test/mail/submit. THE TRAIN every 3 h: 1,000 random tests, merge the passers, run the union (can be every test, 6+ h), attribute non-flaky failures, report. review via ONE GLOBAL APPROVER + pattern tooling.
Q how does review avoid growing with the shard count?

```
   4 CLEAN UP  stop reintroduction at review time
        Q what are the two definitions of "done"?

GROUP 4 - THE DECISION . what deserves removing
-----------------------------------------------------------------
   CODE IS A LIABILITY. functionality is the asset.
        but AGE IS NOT OBSOLESCENCE.
        Q what metric should you maximise instead of lines?
   REMOVAL IS THE ULTIMATE CHANGE  Hyrum's Law: people depend
        on what you never promised. the replacement is better
        AND DIFFERENT, so 1:1 is rare.
   DESIGN FOR IT  can consumers migrate off me? can I be
        replaced in parts? (a power station budgets for its
        own decommissioning.)
   ADVISORY vs COMPULSORY  no deadline / a deadline with
        enforcement AND staffing
        Q what does advisory require of the NEW system?
   WARNINGS  actionable AND relevant, or alert fatigue
   OWNERS  no explicit owner -> no progress. removal must be
        the primary goal. celebrate incremental milestones.
        Q why is the final deletion the least noticed step?
```

SECTION 6 CONTINUED · THE TRANSFERABLE CARD

Master 2×2 — the decision card you can carry to other problems

Two questions, asked at opposite ends of a system's life.

COLUMNS: HAVE YOU COMMITTED TO REMOVING THE OLD ONE? →

	Yes — a deadline, an owner and staffing	**No — it stays until it falls over**
You CAN change every use of it, cheaply	**The old system actually goes away** The mechanism exists and someone is measured on the last reference — the only cell where an old decision can be revisited at will. *Move:* spend the capability before you need it: design successors with a migration path, and let reversibility change what you are willing to decide.	**Capable, and accumulating** You could migrate everyone tomorrow and nobody has decided to, so both stay and the old holds the new back. *Move:* the constraint is not technical — name an owner, pick a few removals, set a date you can enforce.
You CANNOT — changing every use is infeasible	**Committed, and stuck** The intent is real and the work manual, so it moves at the speed of teams with other jobs. *Move:* build the mechanism first — a deadline without one is an unfunded mandate. Frightening code is answered by tests, not tooling.	**Everything is permanent** No way to change the uses and no intent to remove anything, so every decision is a one-way door. *Move:* the cheapest first step is admitting the cost exists — the trap is that doing nothing appears free.

Read it as: the row is a capability you build, the column a decision you make, and neither substitutes for the other. The same pair separates a regulator who can update a form across every council from one issuing guidance, a bank that can retire a payment format from one supporting it forever, and a curriculum body that can change what schools teach from one publishing a recommendation.

THE THUMB RULE

A decision is permanent only if you cannot change every use of it — so build the ability to change them, then decide what to remove. Both halves get skipped, for opposite reasons: the capability helps nobody in particular, and doing nothing has no line item.

SECTION 7

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Planning a large change

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
The commit is rejected by the server, or stalls everyone else.	Technical limitation — version-control operations scale linearly with change size.	Shard it. This is impossibility, not imprudence; accept that execution becomes more complex.
The change is ready and files keep moving underneath it.	Merge conflicts, whose probability grows with files and with engineers.	Fewer files per change. And keep regenerating the parent change as conflicts appear.
Someone proposes doing it at 3 a.m. on a holiday.	The global-lock trick — a weekend, or a token passed round the team.	It works only while somebody is not always committing. Plan for the version where they are.
A subsystem is being excluded from the migration “for safety”.	A haunted graveyard: ancient, obtuse or complex enough that nobody dares enter it.	Write tests for it. Nothing else works, and until it is done it blocks migrations, decommissioning and compiler upgrades for everyone.
A 60-file change is harder than a 9,000-file one.	Heterogeneity, which is governed by the number of environments, not the number of files.	Standardise what you can; ask teams to omit special checks for these shards. The humans get the same benefit as the robots.
A test three layers away in the dependency graph failed.	Transitive testing doing its job — and the farther out, the less likely the failure is yours.	Make the changes smaller so failures are diagnosable. One bad file in 25 is findable; in 10,000 it is not.
The same tests run over and over across shards.	The accepted inefficiency of many small changes.	Leave it. Engineer time hunting failures costs more than the compute — though check that trade-off holds for you.
“We'll be able to do this in one commit once we're bigger.”	The expectation that scale buys atomicity.	Correct it: the largest atomic change gets *smaller* as the codebase and headcount grow.

Running the machine

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Reviewers across the company are complaining about volume.	Generation got cheap; review did not. Nobody protected the reviewers.	Authorisation, then a global approver. This asymmetry is the reason the approval process exists at all.
Shards sit unreviewed for a week.	Review effort is growing with the number of shards.	Assign all shards to one global approver using pattern-based tooling to auto-approve what matches expectation, examining only anomalies.
Local owners approve everything instantly without reading it.	They trust the generating team too much — the usual outcome.	Send them only changes where their review supplies context, not permission. Everything else goes to the global approver.
The script handled most sites and you found more by hand.	Generation is not comprehensive.	Fix the tool. The author must be able to rely on it covering every case they meant to fix.
Generating the change took twice as long in the bigger repository.	Human effort is scaling linearly — you automated the typing.	Rebuild so effort is roughly constant in repository size. Past ~500 edits, the tool wins anyway.
New uses of the old thing appear while you are migrating.	Backsliding — copy-and-paste, long-lived branches, old habits.	Regenerate the parent change, and add review-time flagging so new uses stop arriving.
A shard was rejected and an engineer wants to know why.	Treating a shard as a pet.	Regenerate it. Investigate only if the whole herd is failing.

A project's checks were already red before you arrived.	A preexisting failure, not yours.	Ignore it. Keeping their checks green is the owners' half of the social contract.
A flaky test is blocking submission.	At this scale, low-probability failures are near-certainties.	Ignore recently flaky tests on automated submit, and handle the rare real miss by talking to people.
Testing each shard individually is saturating the cluster.	Isolated runs — very expensive, and off-peak only.	Put them on the train, and group large shards: union first, then the intersection with what failed.
The generated code looks nothing like the surrounding code.	No formatter pass.	Run the language's formatter as a separate pass. Without it this practice never becomes socially acceptable.
The migration reached 100% and the count is creeping back up.	Cleanup was skipped.	Flag new uses of the removed symbol at review time; without that, cleanup never converges.

Deprecating something

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
“It's old, we should replace it.”	Age being mistaken for obsolescence.	Ask what it costs and whether a replacement with comparable functionality exists. Decades-old can still be right.
Two systems do the same job and both are maintained.	The cost of keeping both, which grows over time.	Count the transformation code, the mutual dependence, and the drag the old one puts on the new one's evolution.
Adoption plateaus at a fraction and new uses keep appearing.	Advisory deprecation plus the conceptual pull of existing uses.	Either accept it is advisory, or convert: a deadline you can enforce and a team funded to do the moving.
A deadline exists and each team is told to move itself.	An unfunded mandate with a date on it, which reads as mean spirited.	Staff it. Localise migration expertise in the team removing the system.
The last remaining user says they will not move.	The political hazard — a critical dependency that can veto.	Recognise the deprecation is advisory in fact. Either build the power to enforce, or say so honestly.
Nobody knows who still depends on it.	Dependencies that static analysis cannot see.	Announce planned outages of increasing duration, and rename implementation-only symbols to see who breaks.
Warnings fire tens of thousands of times per build.	Alert fatigue, usually from transitive propagation.	Mark liberally, surface narrowly — newly changed lines only, with intrusive warnings reserved for staffed compulsory efforts.
A warning tells you something is deprecated and nothing else.	It is not actionable.	Name the replacement and the next step, at the level of an average engineer in that area.
The warning arrives weeks after the code was written.	It is not relevant.	Surface it while the engineer is writing the code, and send migration mail months before removal rather than the weekend before.
Everyone agrees it should go and nothing happens.	No explicit owner.	Assign one, with removal as their primary goal. Deprecation loses every priority contest it is entered in.
Four turndowns are running at once and all are late.	The simultaneity limit, for the teams and their customers alike.	Pave some roads, not every road. Choose fewer projects and commit to finishing them.
“I like this code.”	Emotional attachment — understandable, and self-defeating.	Note that removed code remains findable in history, and judge the system on its net cost to the organisation.
The team has nothing to show for six months of work.	Final removal treated as the only milestone.	Set incremental, measurable milestones and celebrate them — deleting a key subcomponent counts.
A wholesale replacement is estimated at two years.	Migration to an entirely new system, whose cost is usually underestimated.	Consider evolving in place instead. It does not avoid deprecation; it breaks it into chunks that pay off sooner.

Reverse mapping — you see a practice, what is it there for?

The other direction. Use this when auditing an organisation's approach rather than planning one change: for each thing you find them doing, say what it was buying and what it does *not* buy.

WHAT YOU FIND PEOPLE DOING	WHAT IT COUNTERS	WHAT IT DOES *NOT* COVER
Sharding a change by project and ownership	Version-control limits and merge conflicts, and failures that cannot be attributed.	Heterogeneity. Smaller pieces do not make four different environments into one.
Writing tests for frightening code	Haunted graveyards — the only thing that dissolves them, whatever the system's age or complexity.	The cost of running those tests, which the transitive model then multiplies.
Standardising checks, tooling and formats	The barrier that stops automation working at all, and the friction humans feel moving between projects.	Whether the change is safe. A consistent untested codebase automates its own breakage.
Funding a central migration team	The unfunded mandate, the repeated first change, and migrations that never fully succeed.	Social acceptance, which comes from a good FAQ and a track record rather than from a budget.
An approval committee	One engineer imposing unbounded review load on everybody else, and indiscriminate tweaking.	Change quality. It advises and escalates; it does not write the tooling.
Change generation with flat human effort	Tooling that quietly becomes a bottleneck as the repository grows, and incomplete coverage.	Review load. Cheap generation is precisely what makes review the next constraint.
A global approver with pattern-based tooling	Review effort growing with the number of shards, and local owners rubber-stamping.	Context. Some changes genuinely need the owning team's knowledge, and those must still go to them.
The test train	The cost of testing every shard in isolation, which is very expensive and off-peak only.	Attribution. A failure still has to be rerun against each change to find its cause.
Ignoring recently flaky tests	Throughput loss from failures that are near-certainties at scale.	A regression visible only through a flake turning into a failure — rare, and handled by talking.
Running the formatter as a separate pass	Machine output that nobody can read afterwards, and generation tooling burdened with style.	Correctness. It makes the change legible, not right.
Review-time flagging of new uses	Backsliding, which otherwise turns cleanup into whack-a-mole.	Existing uses. It stops the inflow; it migrates nothing.
Advisory deprecation	Ignorance of the new system, and the absence of early adopters.	Completion. It will not finish, and it requires the new system to be production-ready with transformative benefits.
Compulsory deprecation	The indefinite maintenance of an obsolete system, and teams deprioritising the work forever.	Anything, without both enforcement and staffing. Missing either, it fails in a specific and predictable way.
Planned outages of increasing duration	Dependents that neither you nor they knew existed.	Static dependencies, where semantic analysis is usually sufficient on its own.
Designing for decommissioning up front	Successor systems that quietly become second systems because nobody can migrate off the first.	The decision to remove, which is separate and still has to be made and funded.

SECTION 8

Numbers worth remembering

NUMBER	WHAT IT IS
4 categories	What large-scale changes are for: cleaning up common antipatterns, replacing uses of a deprecated library feature, enabling low-level infrastructure improvements such as compiler upgrades, and moving users from an old system to a newer one.
5 barriers	Why the atomic version is unavailable: version-control limits, merge conflicts, haunted graveyards, heterogeneity, and testing.
4 phases	The process: authorisation, change creation, shard management, cleanup — with very nebulous boundaries between them.
25 vs 10,000	Finding the root cause of a test failure in a change of 25 files is straightforward; finding one in a 10,000-file change is the proverbial needle in a haystack. This is the argument for small shards in one line.
500 edits	The rule of thumb: past this, it is usually more efficient for an engineer to learn and execute the change-generation tooling than to make the edits by hand. For experienced “code janitors”, often much smaller.
~a dozen	The size of the email list with oversight of the whole large-scale-change process — experienced engineers familiar with the nuances of various languages, plus invited domain experts for the change in question.
10–20%	The double-digit percentage of changes in a project that today result from large-scale changes — meaning a substantial amount of code is changed by people whose full-time job is unrelated to that project.
3 hours / 1,000 tests	The test train is started fresh every three hours, and its first step runs a sample of 1,000 randomly selected tests against each change on it.
6+ hours	How long step 3 can take — running the union of all tests directly affected by the group, which for a large enough or low-level enough change can mean every single test in the repository.
< 1,000 of millions	The vast majority of shards affect fewer than a thousand tests, out of the millions across the codebase. For those affecting more, shards are batched — and because those unions run almost everything anyway, adding more changes to the batch is nearly free.
500,000+ references	The scale of the largest migration attempted to that point: an in-house self-destructing smart pointer, referenced more than half a million times across millions of source files, replaced by the standard-library equivalent introduced in the 2011 revision of the language.
700 × 15,000	At the height of that migration: more than 700 independent changes generated, tested and committed, touching more than 15,000 files, **per day**. Today that throughput is sometimes ten times higher, after refining practices and improving tooling.
2 → 1 alias	How that migration finally ended: by first making the old type an alias of the new one, then performing the textual substitution between old and new, and eventually removing the old alias.
1 character	The pothole worth filling: two header files in the same in-house library whose names used different word separators. Engineers had to remember which was which and found out they had guessed wrong only after a potentially lengthy compile.
~2 weeks	What fixing that separator cost, in background-task effort, with mature tooling — applied without bothering the end users of those files, and measurably reducing the build failures it caused.
2,600+ patches, 50+ people	The outward-facing campaign: after a 2017 vulnerability in a widely used open-source library left anything with it on the transitive classpath open to remote execution, volunteers identified affected projects and sent more than 2,600 patches — with more than fifty humans doing the work instead of automated tooling.
1 billion lines / 3 days	The largest series of such changes ever executed removed more than a billion lines of code over three days — largely an obsolete part of the repository that had been migrated elsewhere.
a few dozen → tens of thousands	The leverage the whole apparatus produces: a few dozen engineers supporting tens of thousands of others, keeping the codebase consistent both spatially and temporally.

2 rejection categories	The only kinds of change the committee has outright rejected: those deemed **dangerous**, and those that are **extremely low value**. As experience grew and costs dropped, the approval threshold dropped too.
2 kinds of deprecation	Advisory — no deadline, not a priority, no dedicated resources — and compulsory, which comes with a deadline and requires both an enforcement mechanism and active staffing.
2 warning properties	Actionable: the reader can perform the next step in practical terms. Relevant: it surfaces when they are actually doing the thing. Without both, warnings accumulate into alert fatigue.
2 design questions	Ask of a new system: how easy will it be for consumers to migrate off it to a potential replacement, and how can parts of it be replaced incrementally?
3 things it changes	The summary claims: the process makes it possible to rethink the immutability of certain technical decisions; traditional models of refactoring break at large scales; and making these changes means making a habit of making them.

Glossary

TERM	MEANING
Large-scale change	Any set of changes that are logically related but cannot practically be submitted as a single atomic unit — because of tooling limits, guaranteed merge conflicts, or repository topology.
Atomic change	A change committed as one indivisible unit. The largest one available to you shrinks as the codebase and the number of engineers grow.
Haunted graveyard	A system so ancient, obtuse or complex that no one dares enter it — often business-critical, frozen in time, and wrapped in bureaucracy. The counter is thorough testing.
Heterogeneity	Variety in version control, continuous integration, project tooling or formatting across an organisation. It defeats automation, because machines need consistent environments to apply the right transformation in the right place.
Transitive testing	Running not only the tests immediately affected by a change but every test that depends on the changed files, however indirectly. Broad coverage, at the price of distant failures that are unlikely to be yours.
Unfunded mandate	Additional requirements imposed by an external entity without balancing compensation. The costs do not disappear; they are paid in fragments by the teams told to comply.
Internalising the externality	What funding a central migration team actually does — taking a cost already being paid diffusely and paying it once, with economies of scale.
Filling potholes	Small codebase-wide fixes that only become worth doing once the machinery exists, and that measurably reduce friction.
Cattle, not pets	The stance toward the individual changes a migration generates: nameless, faceless commits that can be rolled back or rejected at little cost, unless the entire herd is affected.
Backsliding	New uses of the old thing appearing while the migration runs — from copy-and-paste, long-lived branches, or habit. Countered by regenerating the parent change and by review-time flagging.
Sharding	Splitting a global change by project boundary and ownership into pieces that can be submitted atomically, each going through its own test-mail-submit pipeline.
The train model	Testing many changes together on a fixed schedule rather than individually, exploiting the facts that such changes rarely interact and that most affected tests pass for most of them.
Global approver	Someone with ownership rights to approve any change throughout the repository, who reviews shards with pattern-based tooling and examines only the anomalies by hand.
Sublinear human effort	The scaling requirement on change generation: roughly the same human time to produce all required edits regardless of repository size.
Comprehensiveness	The other requirement on generation tooling: that an author can be assured the change covers every case they are trying to fix.

Deprecation	The process of orderly migration away from and eventual removal of obsolete systems.
Advisory deprecation	Deprecation with no deadline, no priority and no dedicated resources — aspirational. Useful for advertising a new system, and it will not finish on its own.
Compulsory deprecation	Deprecation with a deadline, after which dependent systems stop working. Requires both an enforcement mechanism and a specialised team staffed through completion.
Conceptual pull	The tendency of a system's existing uses to attract a large share of new uses, regardless of what the documentation asks for.
Hyrum's Law	The more users a system has, the more of them depend on behaviour it never promised — their use happens to work rather than being guaranteed to work. It is what makes removal the hardest change of all.
Alert fatigue	A term from health care for what happens when warnings accumulate, especially transitively, until users ignore them altogether.
Actionability	A property a warning must have: the reader can actually perform a relevant next step, in practical terms, given the expertise expected of an average engineer.
Relevance	The other property: the warning surfaces at the moment the user performs the indicated action, not long afterwards.
Design for decommissioning	Planning a system's eventual removal while designing it — normal in other engineering disciplines, where funds for decommissioning are allocated up front.
Monorepo	Storing the bulk of an organisation's source in a single repository, with visibility for every engineer into almost all of it — which is what makes codebase-wide analysis and change possible, and what makes review load the binding constraint.
Presubmit check	A check configured to run before a change lands in a project — from dependency allowlists to tests to requiring an associated bug. Useful locally, a source of heterogeneity at scale, and selectively disabled for migration shards.
Domain review	Review of a proposed large change by the owners of the interface being refactored, obtained during authorisation and separate from the oversight body's approval.
Code is a liability	The premise that code carries ongoing cost while the functionality it delivers is the asset — so the metric to maximise is functionality per unit of code.

Answer key

PART 1 · Q1 — THE 40,000-FILE RENAME

(a) A large-scale change is any set of changes that are **logically related but cannot practically be submitted as a single atomic unit**. This qualifies on both of the stated grounds at once: it touches so many files that the underlying tooling cannot commit them all, and it is large enough that it would always have merge conflicts. Note that it qualifies because of the impossibility of atomicity, not because 40,000 is a big number. (b) The first failure is a **technical limitation**: most version control systems have operations that scale linearly with the size of a change, and a system that handles tens of files may lack the memory or processing power to atomically commit thousands — in centralised systems such commits can also block other writers while they process. The remedy is to split the change into smaller independent chunks, accepting that this makes execution more complex. The second is **merge conflicts**: every version control system requires updating and merging if a newer version of a file exists centrally, and the probability of hitting that grows with the number of files in the change, compounded by the number of engineers working in the repository. The remedy is the same shape — with few files in a change, the probability shrinks. (c) The 3 a.m. proposal is the small-company manoeuvre: sneaking in a change that touches every file at a time when nobody is developing, or holding an informal global lock by passing a virtual — or even physical — token around the team. It stops working at exactly the point where **somebody is always making changes to the repository**, which is the normal condition of a large or globally distributed organisation. (d) The billing service is a **haunted graveyard**: a system so ancient, obtuse or complex that no one dares enter it, typically business-critical, frozen in time because any change might make it fail in incomprehensible ways, and often wrapped in layers of bureaucracy to prevent destabilising changes. Skipping it is worse than it looks because these are **anathema to the whole process**: they prevent the completion of large migrations, the decommissioning of other systems that rely on them, and the upgrade of compilers or libraries they use — from this perspective they prevent all kinds of meaningful progress, and the harm is to other people's work rather than to the team that owns them. The one thing that dissolves it is **thorough testing**: when software is thoroughly tested, arbitrary changes can be made to it and you will know with confidence whether they break it, **no matter the age or complexity of the system**.

PART 1 · Q2 — THE 9,000-FILE CHANGE AND THE 60-FILE ONE

(a) The continuous-integration system runs not only the tests immediately impacted by a change but every test that **transitively depends** on the changed files, which gives broad coverage — and the farther away in the dependency graph a test is from the impacted files, **the more unlikely a failure is to have been caused by the change itself**. Two days on a three-layers-away failure is therefore the expected outcome rather than bad luck. What would have prevented it is smaller independent changes: each affects a smaller set of tests, and failures are easier to diagnose and fix. Finding the root cause of a failure in a change of 25 files is straightforward; finding one in a 10,000-file change is the proverbial needle in a haystack. (b) The second migration is hitting **heterogeneity**. These changes work only when the bulk of the effort can be done by computers rather than humans, and computers rely on consistent environments to apply the proper transformations to the correct places; many different version control systems, continuous-integration systems, project-specific tooling or formatting guidelines make sweeping changes difficult. Size does not govern it because the obstacle scales with the number of distinct environments, not the number of files — 60 files spread across several incompatible setups needs several solutions, while 9,000 files in one consistent setup needs one. (c) For the first migration, splitting into 200-file commits is exactly right: it addresses the version-control limits, reduces the conflict probability, and makes failures diagnosable. For the second it accomplishes nothing, because the barrier is not change size; the remedy there is to simplify the environment — embracing some of the complexity by making it standard across the codebase, and asking teams to omit their special checks where the shards touch their project code. Most teams are happy to help, given the benefit. (d) The cost of splitting is that it **makes the execution of the change more complex**. The inefficiency it produces is that **smaller changes cause the same tests to be run multiple times**, particularly tests that depend on large parts of the codebase. It is accepted because **engineer time spent tracking down test failures is much more expensive than the compute time required to run those extra tests** — a conscious decision, and one explicitly flagged as a trade-off that might not hold for every organisation.

PART 2 · Q1 — NINE MONTHS OF ANNOUNCEMENTS

(a) The current approach is an **unfunded mandate**: additional requirements imposed by an external entity without balancing compensation. Even where a new system is categorically better than the one it replaces, **those benefits are often diffused across an organisation** and are therefore unlikely to matter enough for individual teams to want to update on their own initiative — while the cost of updating lands concentrated on each of those teams. Nine months of emails does not change that arithmetic for any single team, so the plateau is the predicted result rather than a surprise. (b) The rise in new call sites follows from the fact that **engineers tend to use existing code as examples when writing new code**, so an old system keeps generating new uses of itself for as long as it exists. The implication for the migration guide is uncomfortable: the guide is not the binding constraint, because it is competing with the pull of the examples already in the repository. A better guide will not fix this, and neither will a third email. (c) Three reasons the platform team should hold the work. First, **the team that built and manages the system also has the domain knowledge** required to fix the references to it; consuming

teams are unlikely to have the context, and it is globally inefficient to expect each of them to relearn expertise that already exists in one place. Second, **centralisation allows faster recovery from errors**, because errors generally fall into a small set of categories and the team running the migration can keep a playbook — formal or informal — for them. Third, **incentives**: having a team with a vested interest in removing the old system responsible for the migration helps ensure it actually gets done, whereas organic migrations are unlikely to fully succeed. The dashboard addresses none of them — it does not move the expertise, it does not build the playbook, and it does not change any consuming team's incentive; it restates the mandate with more visibility, and leaves every team paying the repeated first change of reading the motivation, finding an example, and applying it locally. (d) It is not adding cost. **Funding and staffing such a team is actually just internalising the externalities that an unfunded mandate creates**, with the additional benefit of economies of scale. If the new system is important enough to migrate to, **the costs of migration will be borne somewhere in the organisation** regardless; centralising the migration and accounting for its costs is almost always faster and cheaper than depending on individual teams to migrate organically. The current arrangement has not avoided a cost — it has hidden one.

PART 2 · Q2 — THREE THINGS IN ONE WEEK

(a) The stance is **cattle, not pets**. A pet is the handcrafted change: an engineer may spend days or weeks on its creation, testing and review, comes to know it intimately, and is proud when it is committed. Cattle are the changes a large migration produces — **nameless and faceless commits that might be rolled back or otherwise rejected at any given time with little cost**, often because of an unforeseen problem the tests did not catch or something as simple as a merge conflict. Because automation means tooling can be updated and new changes generated at very low cost, **losing a few cattle now and then isn't a problem**, so the right response to one rejected shard is to regenerate rather than to investigate, and certainly not to halt. The one condition under which the engineer's instinct is right is stated in the same breath: **unless the entire herd is affected** — a failure pattern across many shards is a tooling problem and does deserve a day. (b) The staff engineer is entitled to **question the purpose of a specific commit** being made as part of the broader change, and the author should respond to that comment **just as they would any other review comment**. They are not entitled to stop the programme: it is important that code owners understand the changes happening to their software, and they **do not hold a veto over the broader change**. What earns that arrangement is not authority but evidence — **a good set of frequently asked questions and a solid historic track record of improvements**, plus product teams coming to trust the infrastructure team's domain expertise the way that team trusts theirs. (c) The naming request is worth doing, on the same grounds as the comparable case: two header files in one in-house library used different word separators in their names, which drove consistency purists mad and, more importantly, reduced productivity — engineers had to remember which file used which, and discovered they had got it wrong only after a potentially lengthy compile cycle. Fixing a single-character difference across a codebase that size sounds pointless; mature tooling made it **a couple of weeks' worth of background-task effort**, library authors could apply it **without having to bother end users**, and it **quantitatively reduced the number of build failures** caused by that issue, with productivity and happiness gains that more than paid for the time. (d) The wider consequence is that **engineering decisions can be made knowing they aren't immutable**. As the ability to change the entire codebase improves, the diversity of possible changes expands, and technical decisions that used to be final — the name of a widely used symbol, the location of a popular class — **no longer need to be final**. That is a benefit which applies at design time, before any migration is contemplated.

PART 3 · Q1 — 12,000 SITES AND A WEEKEND SCRIPT

(a) The four phases are **authorisation, change creation, shard management and cleanup**. The plan skipped authorisation entirely. That phase exists because the costs of generating a change and of reviewing it were historically symmetric, which naturally limited what one person could produce — and as tooling improved it became easy to generate a large number of changes very cheaply, and **equally easy for a single engineer to impose a burden on a large number of reviewers across the company**. The process exists to keep **oversight and thoughtfulness** behind such changes rather than indiscriminate tweaking, to help authors produce the best possible change, and to provide an escalation path; it is deliberately lightweight and historically very liberal. (b) The property review must have is that it **does not grow with the number of shards**. The mechanism is the **global approver**: someone with ownership rights to approve any change throughout the repository, to whom all shards are assigned rather than to the owners of individual projects, and who generally has specific knowledge of the language or libraries involved and knows the change's potential failure modes. The way they process 340 shards is not by reading 340 changes: they use **a separate set of pattern-based tooling to review each change and automatically approve those that meet their expectations**, manually examining only the small subset that are anomalous because of merge conflicts or tooling malfunctions. Directing all reviews for a change to a single global approver is in fact the **most common piece of feedback** the oversight committee gives, precisely because first-time authors assume local project owners should review everything. (c) The nine already-failing checks should be **aggressively ignored**. Preexisting failures are not the migration's to fix, and the reason is explicit: **local code owners are responsible for having no preexisting failures in their codebase, as part of the social contract** between them and the infrastructure teams. Fixing them is a technique that works for an engineer on one project and does not scale across a codebase-wide change. (d) The 60 unrecognised sites violate **comprehensiveness**: change-creation tooling should be comprehensive across the codebase, so that an author can be assured their change **covers all of the cases they're trying to fix**. That is answered in the change-creation phase, by fixing the generator rather than by hand-patching. The two new uses are **backsliding**, which is answered in two places — change creation, because the generation process should be automated enough that **the parent change can be updated as users backslide into old uses**, and

cleanup, which requires a system that **prevents additional introductions** of the symbol being removed, typically by flagging new uses of a deprecated object at review time.

PART 3 · Q2 — 2,000 SHARDS, TESTED ONE AT A TIME

(a) They should use the **train model**. It exploits two facts: these changes **rarely interact with one another**, and **most affected tests are going to pass for most of them** — which holds because such changes tend to be pure refactorings, very narrow in scope and preserving local semantics, and because individual changes are simpler and highly scrutinised, so correct more often than not. The train is started fresh **every three hours** and has five steps. **1.** For each change on the train, run a sample of **1,000 randomly selected tests**. **2.** Gather all the changes that passed their thousand and create one uber-change from them — the train. **3.** Run the union of all tests directly affected by the group, which for a large enough or low-level enough change can mean running every single test in the repository, and **can take more than six hours**. **4.** For each non-flaky test that fails, rerun it individually against each change that made it onto the train, to determine which change caused the failure. **5.** Generate a report for each change describing all passing and failing targets, which can be used as evidence that the change is safe to submit. It also has the advantage of working for multiple changes at once without requiring each to ride in isolation — isolated runs can be requested but are very expensive and performed only during off-peak hours. (b) For shards affecting large numbers of tests, group them: **first run the union of all affected tests for all shards, and then for each individual shard run just the intersection of its affected tests with those that failed the first run**. Adding more shards to such a batch is nearly free because **most of these unions cause almost every test in the codebase to be run anyway**. (c) The blocking rule stalls because of a scale effect: one of the drawbacks of running such a large number of tests is that **independent low-probability events are almost certainties at large enough scale**. Flaky and brittle tests often do little harm to the teams that write and maintain them, and are **particularly difficult for the authors of these changes**, seriously affecting throughput. The position taken is that confidence in the correctness of a single semantics-preserving, machine-generated change is now higher than confidence in a test with any recent history of flakiness — **so recently flaky tests are ignored when submitting via the automated tooling**. The residual risk is that in theory a single shard could cause a regression detected only by a flaky test going from flaky to failing; in practice this is seen so rarely that **it is easier to deal with via human communication rather than automation**. (d) Tidying formatting in review puts the cost on reviewers, who are the scarce resource the whole design protects — and it misses the point of the requirement. Because the codebase is optimised for human readability, changes produced by automated tooling must be intelligible to immediate reviewers and to future readers, so **all generation tools run the automated formatter appropriate to the language as a separate pass**, leaving the change-specific tooling free of formatting concerns. Automatically produced changes then “fit in” with human-written code, reducing future development friction. The claim attached is strong: **without automated formatting, large-scale automated changes would never have become the accepted status quo.**

PART 4 · Q1 — EIGHTEEN MONTHS OF WARNINGS

(a) This is **advisory deprecation**: a deprecation that has **no deadline and is not high priority** for the organisation, and for which it is not willing to dedicate resources. Such deprecations are also called **aspirational** — the team knows the system has been replaced and hopes clients will eventually migrate, without imminent plans either to provide support to help them move or to delete the old system — and this kind **often lacks enforcement**. The phrase for the strategy it amounts to: **“hope is not a strategy.”** (b) The 4% growth is predicted by the **conceptual, or technical, pull** that existing uses exert: comparatively many uses of the old system **will tend to pick up a large share of new uses, no matter how much we say “please use the new system”**. The implication for the migration page is that it is not the binding constraint. Putting a deprecation warning on an old system and walking away can lead to slightly fewer new uses of it, but **rarely leads to teams actively migrating away** — so more or better documentation will not change the trajectory. (c) Against the two properties: the warnings may be **actionable** if they name the replacement call and the next step an average engineer in that area could take — but firing at build time in transitive dependents, they are not **relevant**, because relevance means surfacing **at a time when the user actually performs the indicated action**: while the engineer is writing the code that uses the function, not once it has been in the repository for weeks and is being rebuilt by somebody else. The 40,000 transitive firings have produced **alert fatigue**, the term from health care for warnings accumulating until users ignore them altogether — which is exactly what the team filtering their logs is telling them. Two ways to surface narrowly: **limit warnings to newly changed lines**, so they reach the person introducing a new use at the moment they introduce it; and **reserve much more intrusive warnings — such as those on deprecated targets in the dependency graph — for compulsory deprecations where a team is actively moving users**. A second, louder warning makes the condition worse, not better. (d) The precondition is that the benefits of the new system **cannot be simply incremental: they must be transformative**. Users will be hesitant to migrate on their own for marginal benefits, and **even new systems with vast improvements will not gain full adoption using only advisory deprecation efforts**. An 8% serialisation improvement with otherwise equivalent behaviour is squarely incremental, so advisory deprecation will not finish here under any amount of warning. If the team wants it to finish they must convert to **compulsory deprecation** — a deadline after which dependent systems stop working, backed by an enforcement mechanism, and, crucially, **actively staffed by a specialised team through completion** rather than delegated to the consuming teams.

PART 4 · Q2 — THE SCHEDULER THAT WILL NOT DIE

(a) The intent is **compulsory deprecation**: it comes with a deadline for removal of the obsolete system, after which users who still depend on it find their own systems no longer work. It requires two things. It **needs an enforcement mechanism** — not that the schedule can never change, but that the team running the process is empowered to break non-compliant users after they have been sufficiently warned; without that power it becomes easy for customer teams to ignore the work in favour of features. And it must be **actively staffed by a specialised team through completion**, because a compulsory deprecation without staffing comes across as mean spirited and reads to customers as an unfunded mandate requiring them to push aside their own priorities just to keep their services running. The organisation supplied the deadline and the announcement; it supplied neither the staffing — each team was asked to move itself — nor, as events have shown, the enforcement. (b) The hazard in general terms: when **the last remaining user of the old system is a critical piece of infrastructure that the entire organisation depends on**, you have to ask how willing you would be to break that infrastructure, and everything that transitively depends on it, for the sake of an arbitrary deadline. **It is hard to believe the deprecation is really compulsory if that team can veto its progress.** Its existence therefore proves the deprecation was advisory in fact from the start, whatever it was called — which also explains why most teams moving does not indicate success: the tail is the whole problem. (c) Running two turndowns in parallel runs into the stated limit: organisations have **a limit on the amount of deprecation work that is reasonable to undergo simultaneously**, from the point of view of the teams doing the deprecation as well as the customers of those teams. The analogy is road paving: everybody appreciates having freshly paved roads, but **if the public works department decided to close down every road for paving simultaneously, nobody would go anywhere**; by focusing their efforts, paving crews get specific jobs done faster **while also allowing other traffic to make progress**. Hence the instruction to **choose deprecation projects with care and then commit to following through on finishing them** — which is precisely what starting a second turndown two months ago failed to do. (d) Dropping the deadline converts the effort into an **advisory deprecation**, and delegating deprecation to the users of a system **becomes simply an advisory deprecation, which will never organically finish**. What that predicts is the old scheduler maintained indefinitely, continuing to attract new uses through the conceptual pull of its existing ones, while the platform group stays on call for it. The alternative is to make it real: keep a date, ensure the team is **empowered to break non-compliant users after sufficient warning**, and staff the migration — **localising the expertise of migrating users within a single team of experts**, usually the team responsible for removing the old system, which has both the incentive and the accumulated tooling, and using that team to move the reporting pipeline rather than asking it to move itself. On milestones: final removal should not be the only measurable one, since it **might not even happen in all deprecation projects** and, if the team has done its job, is **the least noticed step by anyone external** because by that point the system has no users. Set concrete incremental milestones that are measurable and deliver value, and recognise achievements such as **deleting a key subcomponent** the way you would recognise shipping a new product.

SECTION 10

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 · What makes a change an LSC, if not its size						
2 · The counterintuitive shrink, and why						
3 · The four categories of LSC						
4 · Technical limits and merge conflicts						
5 · Haunted graveyards, and the one counter						
6 · Heterogeneity, and why size doesn't govern it						
7 · Transitive testing, and 25 vs 10,000 files						
8 · The trade accepted: compute vs engineer time						
9 · Unfunded mandate — definition and where cost lands						
10 · The three reasons to centralise						
11 · Why organic migrations stall, mechanically						
12 · Filling potholes — the separator example						
13 · Decisions stop being immutable						
14 · Cattle vs pets, and the one exception						
15 · The social contract: understand, but no veto						
16 · The four phases						
17 · Why authorisation exists, and what it does						
18 · Sublinear generation, and comprehensiveness						
19 · The 500-edit rule of thumb						
20 · Why machine output must look human-written						
21 · Sharding, and what a shard must satisfy						
22 · The train: two facts, five steps, two numbers						
23 · Union-then-intersection batching						
24 · Flaky tests at scale, and the position taken						
25 · Reviewer discovery and preexisting failures						
26 · The global approver, and how they scale						
27 · Cleanup, and preventing reintroduction						
28 · Code is a liability — and what the asset is						
29 · Age is not obsolescence; the cost of keeping both						
30 · Why removal is the ultimate change						
31 · Design for decommissioning — the two questions						
32 · Advisory: what it's for, and its precondition						
33 · Compulsory: the two requirements and the hazard						
34 · Warnings: actionable, relevant, alert fatigue						
35 · Owners and milestones						
36 · Master matrix — can change × committed to remove						

Final quiz — no notes, twenty minutes, write the answers

1. Define a large-scale change without reference to size, state what happens to the largest possible atomic change as a codebase and its engineering population grow, and give the four categories such changes fall into.

2. Name the five barriers to making a change atomically, and give the specific remedy for each — including which barrier is answered by testing and which one testing makes harder.

3. Define a haunted graveyard, say why it is a problem for other people rather than only for its owners, and give the one thing that dissolves it.

4. Define an unfunded mandate, explain in terms of how benefits and costs are distributed why it does not produce migration, and say what funding a central team actually does in economic terms.

5. Give the three reasons the work of a large migration is centralised on the team that owns the system being migrated away from, and state the mechanism by which organic migrations fail to finish.

6. Explain the cattle-and-pets distinction, say what it licenses you to do and under what single condition it does not apply, and state what code owners keep and what they give up under the social contract.

7. Give the four phases of the process, say why the first one exists in terms of the costs of generating and reviewing changes, and give the most common piece of feedback the oversight body offers.

8. State the two scaling requirements on change generation, give the rule of thumb for when to stop editing by hand, and explain why generated changes must be run through an automated formatter.

9. Describe the train model: the two facts it exploits, its five steps with the two numbers attached, and how a global approver reviews a large number of shards without the effort growing with their number.

10. Distinguish advisory from compulsory deprecation, give what each one requires before it is honest to start, and give the two properties a deprecation warning must have and the condition that follows when it lacks them.